

# Dispense di Fondamenti di Intelligenza Artificiale

Marco Galletti

A.A. 2024/2025

## Sommario

Questa dispensa contiene un riassunto dei principali concetti trattati nel corso di Fondamenti di Intelligenza Artificiale con la professoressa Federica Baccin del corso di laurea di "Scienze matematiche per l'intelligenza artificiale" presso l'Università Roma "La Sapienza".

## Indice

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                               | <b>1</b>  |
| 1.1      | Descrizione di ambienti operativi PEAS . . . . .                  | 3         |
| <b>2</b> | <b>Ricerca informata: A*</b>                                      | <b>7</b>  |
| 2.1      | Ricerca best first greedy . . . . .                               | 7         |
| 2.2      | Ricerca A* . . . . .                                              | 8         |
| <b>3</b> | <b>Logica come linguaggio per rappresentare conoscenza</b>        | <b>9</b>  |
| 3.1      | Logica proposizionale . . . . .                                   | 10        |
| 3.1.1    | Tavole di verità . . . . .                                        | 12        |
| <b>4</b> | <b>Conseguenza logica e inferenza</b>                             | <b>13</b> |
| <b>5</b> | <b>Alberi di Beth</b>                                             | <b>15</b> |
| <b>6</b> | <b>Sistema Hilbertiano</b>                                        | <b>16</b> |
| 6.1      | Trasformazione in CNF . . . . .                                   | 19        |
| 6.2      | Sistema di inferenza basato su risoluzione (RES) . . . . .        | 20        |
| 6.3      | Completezza della Risoluzione . . . . .                           | 23        |
| 6.4      | Un algoritmo per la Risoluzione . . . . .                         | 24        |
| 6.5      | Algoritmi di deduzione automatica per clausole definite . . . . . | 25        |
| 6.5.1    | Clausole di Horn . . . . .                                        | 25        |
| 6.5.2    | Concatenazione in avanti per clausole definite . . . . .          | 26        |
| 6.5.3    | Concatenazione all'indietro . . . . .                             | 27        |
| <b>7</b> | <b>Logica del primo ordine</b>                                    | <b>27</b> |
| 7.1      | Sintassi della logica del primo ordine . . . . .                  | 27        |
| 7.2      | Sistema di inferenza per la logica del primo ordine . . . . .     | 33        |
| 7.3      | Sistema Hilbertiano per la logica del primo ordine . . . . .      | 33        |
| 7.4      | Teorie del primo ordine . . . . .                                 | 34        |

## 1 Introduzione

L'intelligenza artificiale è una vasta disciplina che si occupa di costruire sistemi che sono in grado di eseguire compiti che normalmente richiedono intelligenza umana. Essa sfrutta metodi matematici, statistici e di programmazione per imparare da dati e prendere decisioni.

La progettazione di un sistema intelligente ha due tipi di approcci:

- **Approccio simbolico:** si ricorre a tecniche di rappresentazione della conoscenza fatta da regole formali, che specificano come un agente deve agire a fronte delle informazioni che ha a disposizione.
- **Approccio sub-simbolico:** si utilizzano tecniche di apprendimento automatico, che permettono di imparare dai dati e di generalizzare a nuovi problemi.

Una definizione classica dell'intelligenza artificiale è:

L'intelligenza artificiale si occupa di comportamento intelligente negli artefatti. Il comportamento intelligente, a sua volta, coinvolge la percezione, il ragionamento, l'apprendimento, il comunicare e l'agire in ambienti complessi.

—[nilsson1998artificial]

Questi sistemi di intelligenza artificiale agiscono con un ambiente esterno, e riesce a farlo attraverso un sistema di input e output (e.g. sensoristica): una volta immagazzinata l'informazione, questa viene elaborata per creare una rappresentazione formale, che viene poi utilizzata per prendere decisioni.

Concetti chiave che definiscono un agente:

- **Percezione:** dati rilevati in input da sensori in ogni istante temporale (storico completo detto sequenza percettiva)
- **Rappresentazione della conoscenza:** rappresentazione degli stati dell'ambiente da cui può derivare una base di "coscienza" interna
- **Ragionamento:** dall'osservazione dello stato dell'ambiente l'agente è in grado di elaborare una decisione razionale, che segue un "feedback" ottenuto dall'ambiente
- **Azione:** azioni che l'agente può compiere, che ne modificano lo stato e l'ambiente (effettuata attraverso attuatori)

L'architettura, assieme a un programma, definiscono un agente. Un programma agente è il codice che implementa il compostamento di un agente. Un programma specifica l'azione che l'agente deve intraprendere data una certa sequenza percettiva. La funzione agente è la formulazione astratta del programma agente.

$$f : H \rightarrow A$$

dove  $H$  è l'insieme delle sequenze percettive e  $A$  è l'insieme delle azioni.

La funzione agente può essere rappresentata da una tabella di azioni, che associa ad ogni sequenza percettiva l'azione da intraprendere.

**Definizione 1.1** (Programma agente). Sequenza finita di istruzioni che l'agente esegue per trasformare un input in un output. Questa si contrappone alla funzione agente che può essere defomota si un insieme infinito di istruzioni. Il programma agente prende in input solo la percezione corrente, mentre la funzione agente prende in input l'intera sequenza percettiva.

**Definizione 1.2** (Razionalità). Si approssima di seguito la definizione di razionalità. Si parte dalla correttezza della funzione agente: l'agente deve fare l'azione corretta.

- **Misura della performance**
- **Conoscenza a priori sull'ambiente**
- **Possibili azioni**
- **Attuale sequenza di percezioni**

Questi quattro elementi definiscono lo schema PEAS (Performance, Environment, Actuators, Sensors).

**Definizione 1.3.** Un agente si definisce razionale se, per ogni possibile sequenza percettiva, sulla base delle informazioni derivate dalla sequenza percettiva e dalla eventuale conoscenza a priori sull'ambiente, l'agente sceglie l'azione che massimizza il valore atteso della misura della performance.

Parole chiave della razionalità sono:

- **Esplorazione:** l'agente deve esplorare in una certa misura ciò che non conosce, al fine di modificare le percezioni future
- **Apprendimento:** l'agente deve essere in grado di acquisire nuova conoscenza dalla sequenza percettiva
- **Autonomia:** l'informazione derivante dalla conoscenza a priori e dalle sequenze percettive può essere incompleta o scorretta. L'agente razionale deve essere autonomo e "imparare" a compensare conoscenza parziale o errata

## 1.1 Descrizione di ambienti operativi PEAS

Gli ambienti operativi possono essere classificati secondo diverse caratteristiche:

**Gradi di accessibilità dell'ambiente:**

- **Completamente osservabile:** l'agente può osservare l'intero ambiente
- **Parzialmente osservabile:** l'agente può osservare solo una parte dell'ambiente
- **Non osservabile:** l'agente non può osservare l'ambiente

**Numero di agenti:**

- **Unico agente:** l'agente è l'unico agente presente nell'ambiente
- **Multi-agente:** l'agente è uno tra molti agenti presenti nell'ambiente

**Natura della dipendenza tra stati consecutivi:**

- **Deterministico:** l'agente sa esattamente quale sarà lo stato successivo
- **Stocastico:** l'agente sa solo la probabilità di ottenere uno stato successivo

**Struttura dell'esperienza dell'agente:**

- **Sequenziale:** l'agente ha accesso a una sequenza di fotografie passate dell'ambiente e agisce di conseguenza
- **Episodico:** l'agente osserva lo stato corrente, sceglie un'azione, e osserva il nuovo stato

**Variabilità dell'ambiente nel tempo:**

- **Statico:** l'ambiente non cambia nel tempo
- **Dinamico:** l'ambiente cambia nel tempo

**Tipo di rappresentazione di stato, ambiente, tempo e percezioni:**

- **Discreto:** l'ambiente e le percezioni sono discreti
- **Continuo:** l'ambiente e le percezioni sono continui

## Tipi di agenti

- **Agenti reattivi semplici:** l'agente non ha memoria e sceglie l'azione in base alla sola percezione corrente. Questi agenti sono basati su regole-condizione-azione.

if condizione then azione

---

```

1: function AGENTE_REATTIVO_SEMPlice (percezione)
2:   persistent: insieme di regole condizione-azione
3:   stato ← INPUT_INTERPRET (percezione)
4:   regola ← REGOLA_CORRISPONDENTE (stato, regole)
5:   azione ← regola.AZIONE
6:   return azione

```

---

Questi agenti sono limitati, in quanto se l'ambiente non è completamente osservabile è difficile selezionare la decisione corretta sulla base della sola percezione corrente.

- **Agenti reattivi basati su modello:** si introduce lo **stato interno** dell'agente, che permette di memorizzare e aggiornare la conoscenza sull'ambiente esterno, e informazioni su come le azioni dell'agente producano cambiamenti nell'ambiente.

La loro limitazione sta nel fatto che tenere traccia dello stato corrente dell'ambiente e dell'effetto delle proprie azioni su di esso può non essere sufficiente per prendere decisioni corrette.

- **Agente basati su obiettivi:** sono caratterizzati dalla specificazione di uno o più obiettivi, che influenzano la scelta delle azioni da intraprendere. La difficoltà di questi agenti sta nel capire quale sequenza di azioni raggiungerà l'obiettivo. Hanno il vantaggio che la conoscenza può essere rappresentata esplicitamente e può essere modificata dinamicamente, tuttavia richiede operazioni costose dal punto di vista computazionale (ricerca, pianificazione). Possono inoltre esistere obiettivi conflittuali.
- **Agente basati su utilità:** per gestire la conflittualità degli obiettivi, si utilizza una funzione di utilità, che associa un valore ad ogni possibile azione. Si vuole massimizzare l'utilità attesa.

Vi sono agenti con la capacità di apprendere, che possono modificare la propria funzione agente in base all'esperienza. L'ambiente invece può essere rappresentato come:

- **Rappresentazione atomica:** ogni stato del mondo è indivisibile. È una rappresentazione molto astratta.
- **Rappresentazione fattorizzata:** uno stato è un vettore di lunghezza fissata, le cui componenti sono i valori delle corrispondenti variabili. Gli stati possono differire per il valore di una o più variabili. È una rappresentazione meno astratta rispetto alla rappresentazione atomica.
- **Rappresentazione strutturata:** stato è una struttura complessa che descrive relazioni tra gli oggetti in un determinato ambiente. È il livello di astrazione più basso. Sono alla base di Database relazionali e logica del primo ordine.

È interessante cogliere come avviene la selezione di azioni da parte di un agente. Un agente può:

- Effettuare una sequenza di azioni per raggiungere un obiettivo (agenti risolutori di problema)
- Utilizzare processi interni di "ragionamento" per acquisire conoscenza e derivare azioni da intraprendere (agenti basati su conoscenza)

**Agenti risolutori di problema (PS)** Un agente PS sfrutta una rappresentazione atomica degli stati dell'ambiente per formulare un problema come una ricerca nello spazio degli stati. Possiamo strutturare il paradigma degli agenti PS come:

1. Formulazione dell'obiettivo: definizione degli stati che caratterizzano il raggiungimento dell'obiettivo
2. Formulazione del problema: definizione rappresentazione di ogni stato e di transizioni di stato.
3. Ricerca: trovare una soluzione (sequenza di azioni) attraverso un processo di ricerca
4. Esecuzione: esegue le azioni per arrivare alla soluzione

**Osservazione 1.1.** Una volta trovata una soluzione l'agente può fare a meno delle "percezioni" e raggiungere l'obiettivo.

**Definizione 1.4** (Problema di ricerca). Dato  $S = \{S_0, S_1, \dots, S_n\}$  lo spazio degli stati, uno stato iniziale  $S_i \in S$ , un insieme di stati finali  $\{S_0^f, S_1^f, \dots, S_m^f\} \subseteq S$ , un insieme di azioni  $A = \{a_1, a_2, \dots, a_k\}$  e una funzione:

$$\text{ACTION} : S \rightarrow \mathcal{P}(A)$$

che associa ad ogni stato l'insieme delle azioni che possono essere eseguite in quello stato. Necessitiamo anche di un modello di transizione che descriva come ogni azione modifica l'ambiente:

$$\text{RESULT} : S \times A \rightarrow S$$

che associa ad ogni coppia (stato, azione) lo stato che ne risulta. Definiamo la funzione di costo dell'azione:

$$\text{COST\_ACTION} : S \times A \times S \rightarrow \mathbb{R}$$

che associa ad ogni tripletta  $(S_{j1}, a_j, S_{j2})$ , dove  $S_{j2}$  è uno stato raggiunto da  $S_{j1}$  mediante l'azione  $a_j$ .

**Definizione 1.5** (Soluzione di un problema di ricerca). Spazio degli stati può essere rappresentato come un **grafo pesato**  $G = \langle V, E, W \rangle$ , in cui i vertici  $V$  rappresentano gli stati, le etichette  $E$  rappresentano le azioni, e la funzione peso  $W : E \rightarrow \mathbb{R}$  associa ad ogni arco il costo dell'azione corrispondente. Una soluzione di un problema di ricerca corrisponde a un cammino  $v_0^c, v_1^c, \dots, v_k^c$  in cui  $v_0^c \in S_i$  e  $v_k^c \in \{S_0^f, S_1^f, \dots, S_m^f\}$ . Un cammino  $C$  di lunghezza  $m$  su un grafo è una sequenza di vertici  $v_0^c, v_1^c, \dots, v_k^c$  tali che per ogni  $i = 0, 1, \dots, k-1$  esiste un arco  $(v_i^c, v_{i+1}^c) \in E$ . Una soluzione ottima tra tutte le soluzioni è quella che minimizza il costo totale del cammino. (Algoritmi di ricerca su grafi trovano soluzioni ottime)

**Valutazione della prestazione di un PS** I criteri che ci permettono di scegliere un certo algoritmo di ricerca sono:

- **Completezza:** l'algoritmo trova sempre una soluzione se esiste
- **Ottimalità:** l'algoritmo trova la soluzione al problema di ricerca con il minimo costo
- **Complessità temporale:** tempo necessario per trovare una soluzione (in funzione del numero di stati)
- **Complessità spaziale:** memoria necessaria per effettuare la ricerca nello spazio degli stati

**Limiti degli agenti PS** Gli agenti PS possiedono una conoscenza del mondo esterno limitata a ciò che serve per raggiungere gli obiettivi, e non acquisisce informazioni aggiuntive sull'ambiente. Non sono quindi in grado di fare deduzioni. Questo problema è dato dall'utilizzo di rappresentazioni atomiche.

L'idea che ne deriva è quella di utilizzare un approccio più sofisticato per avere un livello di astrazione più alto, per poi applicare processi di inferenza per derivare azioni da intraprendere.

### Agenti basati su conoscenza

**Definizione 1.6.** Un agente basato su conoscenza sfrutta una rappresentazione interna della conoscenza per formulare un processo di ragionamento a seguito del quale deriva azioni da intraprendere.

I suoi componenti sono:

- **Base di conoscenza:** insieme di formule che rappresentano asserzioni sull'ambiente
- **Sistema di inferenza:** regole che determinano come l'agente può derivare nuove asserzioni dalla base di conoscenza
- **Motore di ragionamento:** algoritmo che utilizza le regole di inferenza per derivare nuove asserzioni dalla base di conoscenza

A differenza di un agente PS, un agente KB non necessariamente conosce tutti gli stati e le azioni che possono essere eseguite, ma può derivare nuove asserzioni dalla base di conoscenza.

Un agente basato su conoscenza prevede meccanismi per:

- **Aggiungere nuove formule** alla base di conoscenza  $\rightarrow$  TELL
- **Interrogare** la base di conoscenza  $\rightarrow$  ASK

---

```

1: function AGENTE_REATTIVO_SEMPLICE (percezione)
2:   persistent: insieme di regole condizione-azione
3:   stato  $\leftarrow$  INPUT_INTERPRET (percezione)
4:   regola  $\leftarrow$  REGOLA_CORRISPONDENTE (stato, regole)
5:   azione  $\leftarrow$  regola.AZIONE
6:   return azione

```

---

### Funzioni utilizzate:

- INPUT\_INTERPRET(*percezione*): interpreta la percezione in ingresso e restituisce lo *stato*.
- REGOLA\_CORRISPONDENTE(*stato*, *regole*): seleziona la regola condizione-azione applicabile allo *stato*.
- AZIONE: indica l'azione da eseguire.

Lo schema non è direttamente implementabile come programma agente al fine del calcolo delle azioni: ASK e TELL non possono essere direttamente descritte senza fare riferimento alla struttura della base di conoscenza.

**Osservazione 1.2.** *A differenza degli agenti risolutori di problemi che sono caratterizzati a livello implementativo, gli agenti basati su conoscenza sono definiti e formalizzati al livello della rappresentazione della conoscenza stessa.*

**Approccio dichiarativo** Agente basato su conoscenza costruito comunicandogli con l'operazione TELL quello che deve sapere. Un progettista può partire da una Knowledge Base (KB) vuota e comunicare un insieme di informazioni al fine di costruire una base di conoscenza che l'agente necessita per operare in autonomia.

Questo approccio ha il vantaggio di essere flessibile.

**Approccio procedurale** Un agente basato su conoscenza è costruito codificando direttamente nel programma i comportamenti desiderati. Il vantaggio di questo approccio è che l'agente è da subito in grado di compiere azioni sulla base delle istruzioni fornitegli.

Solitamente si utilizza un approccio ibrido, che comprende sia la codifica diretta della KB sia la possibilità di arricchire la base di conoscenza per acquisire nuove informazioni.

## 2 Ricerca informata: A\*

Per effettuare una ricerca nello spazio degli stati, si può utilizzare una ricerca informata. La formulazione del problema richiede:

- **Spazio degli stati**
- **Stato iniziale**
- **Obiettivo**
- **Azioni:** associate allo stato di riferimento
- **Modello di transizione**
- **Funzione di costo**

Un albero di ricerca è una struttura dati che descrive **cammini tra stati** per raggiungere un obiettivo. L'albero di ricerca può comprendere vari cammini per raggiungere un qualunque obiettivo. Ma per ogni nodo **dell'albero** esiste un solo cammino per tornare alla radice.

Un algoritmo di ricerca richiede una struttura dati che consenta di tenere traccia dell'albero di ricerca. Dato un **nodo**  $n$ , si definisce:

- $n.state$ : rappresenta lo stato associato al nodo
- $n.padre$ : rappresenta il nodo padre del nodo corrente
- $n.azione$ : rappresenta l'azione che ha portato al nodo corrente
- $n.costo\_cammino$ : rappresenta il costo del cammino per raggiungere il nodo corrente

Definiamo poi la **frontiera**  $frontiera$  su cui sono definite le operazioni:

- $Vuota(frontiera)$ : restituisce **true** se la frontiera è vuota, **false** altrimenti
- $Pop(frontiera)$ : estrae un nodo dalla frontiera
- $Top(frontiera)$ : restituisce il nodo in cima alla frontiera
- $Push(frontiera, nodo)$ : aggiunge un nodo alla frontiera

Ricordiamo che abbiamo 3 code nella ricerca: la coda con priorità, la coda FIFO (first in first out) e la coda LIFO (last in first out).

Definiamo la funzione di valutazione  $f(n)$  che determina la scelta di espandere determinati nodi dell'albero di ricerca. Ad esempio per la ricerca in ampiezza  $f(n)$  è la profondità dei nodi, per la ricerca in profondità  $f(n)$  fa sì che venga espanso il nodo più profondo.

A differenza degli algoritmi di ricerca non informati la ricerca informata sfrutta una **conoscenza specifica** del dominio. La funzione di valutazione dipende da una funzione di **euristica**  $h(n)$  che esprime la distanza dall'obiettivo.

### 2.1 Ricerca best first greedy

La ricerca best first predilige l'espansione di nodi della frontiera su cui la **funzione di valutazione**  $f(n)$  assume il valore minimo (potenzialmente più vicini all'obiettivo). Data una euristica  $h(n)$  la ricerca best first greedy espande prima il nodo con valore più basso dell'euristica:

$$f(n) = h(n)$$

**Osservazione 2.1.** • *Non sempre l'algoritmo restituisce la soluzione ottima*

- *L'algoritmo è complesso se lo spazio degli stati è finito*
- *Complessità spaziale:  $O(|V|)$*
- *Complessità temporale:  $O(|V|)$*

## 2.2 Ricerca A\*

L'algoritmo di ricerca A\* seleziona i nodi da espandere sulla base della seguente valutazione:

$$f(n) = g(n) + h(n)$$

Con  $h(n) \geq 0$  e  $h(\text{obiettivo}) = 0$ .

- $g(n)$ : costo del cammino dallo stato iniziale per raggiungere il nodo  $n$
- $h(n)$ : euristica **ammissibile**: stima del costo per raggiungere lo stato obiettivo da  $n$

Una proprietà importante in una euristica è quella di ammissibilità

**Definizione 2.1.** Una euristica  $h(n)$  si dice ammissibile se per ogni nodo  $n$  non obiettivo si ha:

$$h(n) \leq h^*(n)$$

dove  $h^*(n)$  è il costo effettivo del cammino minimo da  $n$  allo stato obiettivo e  $h(n)$  è una stima di tale costo. Una euristica ammissibile non sovrastima mai il costo del cammino minimo.

### Ottimalità di A\*

**Teorema 2.1.** La ricerca A\* è ottima rispetto al costo

*Dimostrazione.* Procediamo per assurdo. Sia  $C$  il costo del cammino restituito dall'algoritmo A\*. Supponiamo che non sia ottimo, cioè esista un altro cammino di costo minore:  $C^* < C$ .

Allora deve esistere un nodo  $n$  che sta sul cammino di costo  $C^*$  e che non è stato espanso dall'algoritmo (altrimenti sarebbe stato scoperto il cammino ottimo).

Sia  $g^*(n)$  il costo del cammino ottimo dallo stato iniziale a  $n$  e sia  $h^*(n)$  la stima del costo del cammino ottimo da  $n$  fino all'obiettivo più vicino.

Per giungere a una contraddizione, consideriamo le seguenti disuguaglianze:

$$\begin{array}{ll} f(n) > C^* & [\text{se così non fosse, } n \text{ sarebbe stato espanso}] \\ f(n) = g^*(n) + h(n) & [\text{per definizione di } f(n), \text{ e poiché } n \text{ è sul cammino ottimo}] \\ h(n) \leq h^*(n) & [\text{ipotesi di ammissibilità di } h] \\ \implies f(n) \leq g^*(n) + h^*(n) = C^*. \end{array}$$

Ma ciò contraddice la precedente ipotesi  $f(n) > C^*$ . Pertanto la supposizione iniziale ( $C$  non ottimo) è falsa e il cammino trovato da A\* deve essere ottimo.  $\square$

**Osservazione 2.2.** • Una euristica ammissibile, sebbene sottostimi la distanza dall'obiettivo, ci fa trovare il cammino migliore

- Una euristica che sovrastima la distanza dall'obiettivo può tagliare dei rami utili
- La funzione  $g$  ci fa evitare cammini che vadano troppo in profondità

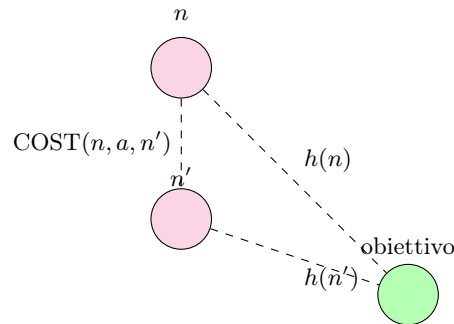
### Consistenza di una euristica

Una euristica  $h$  si dice consistente se per ogni nodo  $n$  e ogni successore  $n'$  si ha:

$$h(n) \leq c(n, a, n') + h(n')$$

dove  $c(n, a, n')$  è il costo dell'azione  $a$  da  $n$  a  $n'$ . La **consistenza** è una proprietà più forte dell'ammissibilità. Ricorda un'uguaglianza triangolare.





**Teorema 2.2.** *Una euristica consistente è ammissibile.*

**Corollario 2.1.** *La ricerca  $A^*$  con una euristica consistente è ottima rispetto al costo.*

#### Proprietà fondamentali $A^*$

- La ricerca  $A^*$  è completa se l'euristica è ammissibile
- La ricerca  $A^*$  è ottima se l'euristica è consistente
- La ricerca  $A^*$  è ottimamente efficiente, ovvero ogni altro algoritmo che utilizza la stessa euristica espande tutti i nodi che sono espansi da  $A^*$ .

#### Limiti di $A^*$

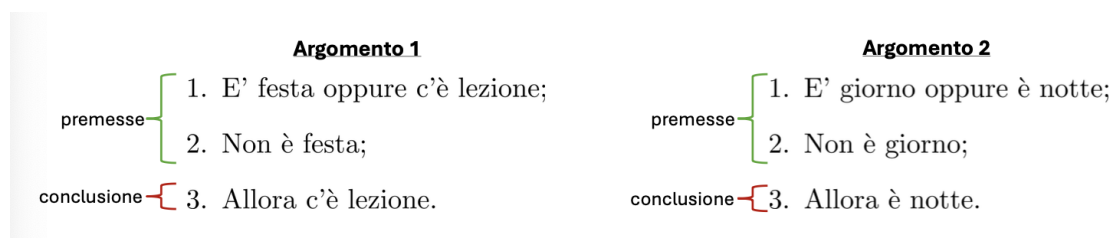
- Il numero di nodi espansi può aumentare esponenzialmente con la lunghezza della soluzione
- Nella pratica si possono adottare varianti della ricerca  $A^*$  che permette di espandere un numero minore di nodi (ricerca  $A^*$  pesata)
- Esistono altri algoritmi che introducono strategie per contenere problema della memoria (ricerca con memoria limitata, IDA\*)

## 3 Logica come linguaggio per rappresentare conoscenza

Ci occuperemo della **logica classica** che è utilizzata per rappresentare una conoscenza dichiarativa ed effettuare ragionamenti formali a partire da essa. La conoscenza dichiarativa riguarda la rappresentazione di **fatti** e **relazioni** che esistono tra oggetti tramite asserzioni vere o false. Per codificare la conoscenza e definire le regole del ragionamento si deve specificare un **linguaggio formale**.

**Definizione 3.1** (Logica). La logica fornisce un sistema di concetti e strumenti che permettono di costruire modelli rigorosi di vari tipi d'inferenza e dei concetti di interpretazione, ragionamento e regola.

La **forma logica** riguarda la strutturazione di enunciati e ragionamenti in un linguaggio formale. Un **ragionamento** è un insieme di enunciati connessi tra loro tramite **inferenze**.



La forma logica comune a questi due argomenti è:

$$(A \vee B), \neg A \implies B$$

**Sillogismo ipotetico:**

$$A \Rightarrow B, B \Rightarrow C \Rightarrow A \Rightarrow C$$

## Enunciati atomici della logica classica

Gli elementi di base che costituiscono il discorso dichiarativo sono detti **enunciati atomici**. Un enunciato atomico è una proposizione che non può essere ulteriormente scomposta in proposizioni più semplici. Gli enunciati atomici assumono una forma logica diversa a seconda del linguaggio formale utilizzato. Distinguiamo tra:

- **Logica proposizionale:** unica formula atomica  $A$
- **Logica del primo ordine:** predicato monadico della forma  $P(f)$  dove  $f$  fa parte del contesto del discorso e  $P$  denota una relazione che sussiste nel sistema di riferimento

### 3.1 Logica proposizionale

**Simboli del linguaggio:**

- $\{A_1, A_2, \dots, A_n\}$ : variabili proposizionali
- $\perp, \top$ : costanti proposizionali
- **Simboli logici:**  $\wedge$  (AND),  $\vee$  (OR),  $\neg$  (NOT),  $\Rightarrow$  (implicazione),  $\Leftrightarrow$  (doppia implicazione)
- **Simboli ausiliari:** parentesi tonde e quadre

Le formule proposizionali **ben formate** sono tutte quelle combinazioni che sono consentite in un linguaggio.

**Definizione 3.2** (Formule proposizionali). 1. Ogni variabile proposizionale è una formula (enunciati atomici)

2.  $\top$  e  $\perp$  sono formule

3. Se  $\alpha$  è una formula, allora anche  $\neg\alpha$  è una formula

4. Se  $\alpha$  e  $\beta$  sono formule, allora anche  $\alpha \wedge \beta$ ,  $\alpha \vee \beta$ ,  $\alpha \Rightarrow \beta$ ,  $\alpha \Leftrightarrow \beta$  sono formule

5. Nient'altro è una formula

**Convenzioni sulle parentesi:**

- Si omettono le parentesi più esterne
- Si stabilisce il seguente ordine di precedenza:  $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
- Si omettono le parentesi che sono superflue sulla base dell'ordine di precedenza
- Data una formula proposizionale complessa questa ha una delle strutture seguenti:
  - è della forma  $\neg\beta$
  - è della forma  $\beta \circ \gamma$  dove  $\circ$  è uno dei connettivi logici e  $\beta$  e  $\gamma$  sono univocamente determinate. Il connettivo  $\circ$  si dice **principale**

**Definizione 3.3** (Semantica). La semantica di un linguaggio attribuisce uno e uno solo stato di verità alle formule di un linguaggio. La semantica della logica proposizionale stabilisce il significato dei **connettivi logici** e di conseguenza delle formule della logica proposizionale. Si considerano tre principi che caratterizzano la semantica della logica proposizionale:

- **Principio di determinatezza:** un enunciato dichiarativo si trova in uno e uno solo stato di verità
- **Principio di bivalenza:** ogni enunciato dichiarativo è vero o falso

- **Principio di estensione:** lo stato di verità di una formula è determinato dal valore di verità degli enunciati atomici che la compongono e sul significato dei connettivi logici

**Condizioni di verità per i connettivi logici:**

- **Negazione:**  $\neg A$  è vera se e solo se  $A$  è falsa

| $A$ | $\neg A$ |
|-----|----------|
| 0   | 1        |
| 1   | 0        |

- **Congiunzione:**  $A \wedge B$  è vera se e solo se  $A$  e  $B$  sono vere

| $A$ | $B$ | $A \wedge B$ |
|-----|-----|--------------|
| 1   | 1   | 1            |
| 1   | 0   | 0            |
| 0   | 1   | 0            |
| 0   | 0   | 0            |

- **Disgiunzione:**  $A \vee B$  è vera se e solo se almeno una delle due formule è vera

| $A$ | $B$ | $A \vee B$ |
|-----|-----|------------|
| 1   | 1   | 1          |
| 1   | 0   | 1          |
| 0   | 1   | 1          |
| 0   | 0   | 0          |

- **Implicazione:**  $A \implies B$  è vera se e solo se  $A$  è falsa o  $B$  è vera

| $A$ | $B$ | $A \implies B$ |
|-----|-----|----------------|
| 1   | 1   | 1              |
| 1   | 0   | 0              |
| 0   | 1   | 1              |
| 0   | 0   | 1              |

- **Bicondizionale:**  $A \iff B$  è vera se e solo se  $A$  e  $B$  hanno lo stesso valore di verità

| $A$ | $B$ | $A \iff B$ |
|-----|-----|------------|
| 1   | 1   | 1          |
| 1   | 0   | 0          |
| 0   | 1   | 0          |
| 0   | 0   | 1          |

**Osservazione 3.1.** *I connettivi della logica proposizionale possono essere ridefiniti utilizzando solamente  $\neg$  e  $\implies$ :*

- $A \wedge B \equiv \neg(A \implies \neg B)$
- $A \vee B \equiv \neg A \implies B$
- $A \iff B \equiv (A \implies B) \wedge (B \implies A)$

**Definizione 3.4** (Modello). Un modello  $\mathcal{M}$  per una formula è un'assegnazione di valori di verità agli enunciati atomici che compongono la formula. Si dice che una assegnazione  $\mathcal{M}$  è un modello di una formula  $\alpha$  se la formula  $\alpha$  è vera in  $\mathcal{M}$ .

Un modello è dunque una funzione che associa un valore booleano ad ogni enunciato atomico. Se denotiamo un modello come:

$$\mathcal{M} : P \rightarrow \{0, 1\}$$

$$\mathcal{M} \models \alpha : \alpha \text{ è vera in } \mathcal{M}$$

$\mathcal{M} \models A$  è definito per induzione su  $A$  come segue:

1.  $\mathcal{M} \models p$  se e solo se  $\mathcal{M}(p) = 1$
2.  $\mathcal{M} \models \top$  e  $\mathcal{M} \not\models \perp$
3.  $\mathcal{M} \models \neg \alpha \iff \mathcal{M} \not\models \alpha$
4.  $\mathcal{M} \models \alpha \wedge \beta \iff \mathcal{M} \models \alpha \text{ e } \mathcal{M} \models \beta$
5.  $\mathcal{M} \models \alpha \vee \beta \iff \mathcal{M} \models \alpha \text{ o } \mathcal{M} \models \beta$
6.  $\mathcal{M} \models \alpha \implies \beta \iff \mathcal{M} \not\models \alpha \text{ o } \mathcal{M} \models \beta$
7.  $\mathcal{M} \models \alpha \iff \beta \iff \mathcal{M} \models \alpha \text{ e } \mathcal{M} \models \beta \text{ oppure } \mathcal{M} \not\models \alpha \text{ e } \mathcal{M} \not\models \beta$

Se un assegnazione  $\mathcal{M}$  non è un modello di  $A$  si dice che è un **contramodello** di  $A$  e si denota con  $\mathcal{M} \not\models A$ . Una formula  $A$  è **soddisfacibile** se esiste un modello  $\mathcal{M}$  che la rende vera.

$$\mathcal{M} \models A : \mathcal{M} \text{ soddisfa } A$$

### 3.1.1 Tavole di verità

Attraverso la costruzione di una tavola di verità è possibile assegnare un valore di verità a una formula.

#### • ESEMPIO

$$\neg A \rightarrow (A \rightarrow B)$$

1. Si individua il connettivo principale  $\neg A \rightarrow (A \rightarrow B)$
2. Si assegnano i possibili valori agli enunciati componenti  $\neg A \rightarrow (A \rightarrow B)$
3. Si procede assegnando dall'interno all'esterno i valori di verità ai connettivi seguendone le tavole di verità, fino ad arrivare al connettivo principale.

| $\neg$ | $A$ | $\rightarrow$ | $(A$ | $\rightarrow$ | $B)$ |
|--------|-----|---------------|------|---------------|------|
| 0      | 1   |               | 1    | 1             | 1    |
| 0      | 1   |               | 1    | 0             | 0    |
| 1      | 0   |               | 0    | 1             | 1    |
| 1      | 0   |               | 0    | 1             | 0    |

→

| $\neg$ | $A$ | $\rightarrow$ | $(A$ | $\rightarrow$ | $B)$ |
|--------|-----|---------------|------|---------------|------|
| 0      | 1   | 1             | 1    | 1             | 1    |
| 0      | 1   | 1             | 1    | 0             | 0    |
| 1      | 0   | 1             | 0    | 1             | 1    |
| 1      | 0   | 1             | 0    | 1             | 0    |

**OSSERVAZIONE:** il punto 2 può essere sistematizzato nel caso in cui ci siano  $A_1, \dots, A_n$  enunciati atomici scrivendo  $2^{n-1}$  valori uguali a 1 sotto la colonna di  $A_1$  e i rimanenti valori uguali a 0; in generale per  $A_k$  si scrivono alternativamente nella colonna corrispondente  $2^{n-k}$  copie di 1 e  $2^{n-k}$  copie di 0 fino a esaurimento righe.

Il connettivo principale di una formula è quello che determina il valore di verità della formula.

**Definizione 3.5** (Tautologia). Una **tautologia** è una formula che è vera per ogni assegnazione dei valori di verità degli enunciati atomici che la compongono. Equivalentemente, una tautologia è una formula che è vera in ogni modello (**formula valida**).

**Esempio 3.1.** • **Principio di non-contraddizione:**  $\neg(A \wedge \neg A)$

- **Principio del terzo escluso:**  $A \vee \neg A$
- **Legge di De Morgan I:**  $\neg(A \vee B) \iff (\neg A \wedge \neg B)$

- **Legge di De Morgan II:**  $\neg(A \wedge B) \iff (\neg A \vee \neg B)$

Formalizzare un discorso dichiarativo significa tradurre espressioni dichiarative del linguaggio naturale nel linguaggio logico scelto.

## 4 Conseguenza logica e inferenza

**Definizione 4.1** (Conseguenza logica). Date due formule proposizionali  $F_1$  e  $F_2$ , si dice che  $F_2$  è **conseguenza logica** di  $F_1$  (o che  $F_2$  segue logicamente da  $F_1$ ), se e solo se  $F_2$  è vera in ogni modello di  $F_1$ . Si indica con:

$$F_1 \models F_2$$

Equivalentemente, questo significa che:

$$\mathcal{M}(F_1) \subseteq \mathcal{M}(F_2)$$

Ossia, ogni modello di  $F_1$  è anche un modello di  $F_2$ . In altri termini, diciamo che  $F_1$  è più "forte" di  $F_2$  perché esclude più modelli.

**Definizione 4.2** (Inferenza corretta). Un'inferenza è **corretta** se e solo se la formula conclusiva è conseguenza logica delle premesse. Equivalentemente, ciò significa che ogni modello delle premesse è anche modello della conclusione.

**Esempio 4.1** (Inferenza corretta: sillogismo). Consideriamo le formule:

$$A \rightarrow B, \quad B \rightarrow C \models A \rightarrow C$$

Tale inferenza è corretta perché ogni modello che verifica le premesse  $A \rightarrow B$  e  $B \rightarrow C$ , verifica necessariamente anche la conclusione  $A \rightarrow C$ .

**Teorema 4.1** (di deduzione). Siano date le formule  $A_1, \dots, A_n$  e  $A$ . Allora:

$$A_1 \wedge A_2 \wedge \dots \wedge A_n \models A \iff (A_1 \wedge \dots \wedge A_n) \rightarrow A \text{ è una tautologia}$$

Diciamo in questo caso che:

$$\frac{A_1 \wedge \dots \wedge A_n}{A}$$

è un' **inferenza corretta**.

*Dimostrazione.* Procediamo per doppia implicazione.

( $\Rightarrow$ ) Supponiamo per assurdo che esista  $\mathcal{M}$  tale che:

$$\mathcal{M} \not\models (A_1 \wedge \dots \wedge A_n) \rightarrow A$$

Ossia  $\mathcal{M} \models (A_1 \wedge \dots \wedge A_n)$  e contemporaneamente  $\mathcal{M} \not\models A$ , ma questo contraddice l'ipotesi, dunque abbiamo raggiunto un assurdo.

( $\Leftarrow$ )  $A_1 \wedge \dots \wedge A_n \rightarrow A$  è una tautologia, per cui è vera in ogni modello. Se  $\mathcal{M} \models (A_1 \wedge \dots \wedge A_n) \rightarrow A$ , allora per definizione di implicazione logica, ogni volta che  $\mathcal{M} \models (A_1 \wedge \dots \wedge A_n)$  si ha anche  $\mathcal{M} \models A$ . Quindi  $A$  segue logicamente dalle premesse.  $\square$

**Teorema 4.2.** Date due formule proposizionali  $\alpha$  e  $\beta$ , e

$$\alpha \models \beta \iff \alpha \rightarrow \beta \text{ è una tautologia}$$

**Esempio 4.2** (Mondo del Wumpus). Consideriamo il dominio del Wumpus con il seguente insieme di variabili proposizionali:

$$\mathcal{P} = \{P_{xy}, W_{xy}, B_{xy}, S_{xy}, L_{xy}\}$$

dove:

- $P_{xy}$ : pozzo nella stanza  $[x, y]$
- $W_{xy}$ : Wumpus nella stanza  $[x, y]$
- $B_{xy}$ : brezza nella stanza  $[x, y]$
- $S_{xy}$ : fetore nella stanza  $[x, y]$
- $L_{xy}$ : l'agente è nella stanza  $[x, y]$

**Base di conoscenza (KB)** iniziale:

- $R1 : \neg P_{11}$
- $R2 : B_{11} \leftrightarrow (P_{12} \vee P_{21})$
- $R3 : B_{21} \leftrightarrow (P_{11} \vee P_{22} \vee P_{31})$
- $R4 : B_{11}$

Supponendo che l'agente si sposti in  $[2, 1]$  e percepisca una brezza, la KB aggiornata è:

- $R5 : B_{21}$

**Domanda:** Come si deriva  $\neg P_{12}$  dalla KB aggiornata? **Risposta:** usando metodi di **model checking** (verifica del modello).

**Model checking** Metodo di deduzione basato sull'enumerazione di tutti i modelli della KB tramite tavole di verità, per verificare che la formula desiderata sia vera in ogni modello in cui la KB è vera. Questo metodo è equivalente alla verifica che  $\alpha \rightarrow \beta$  sia una tautologia, per determinare se  $\alpha \models \beta$ .

**Procedimento del model checking:**

1. Enumerare tutti i modelli della KB.
2. Verificare che la formula di interesse sia vera in ciascuno di questi modelli.

Caratteristiche principali:

- **Correttezza:** implementa direttamente la definizione di conseguenza logica.
- **Completezza:** tutte le conseguenze logiche possono essere verificate.
- **Complessità:**  $O(2^n)$ , dove  $n$  è il numero di enunciati atomici (molto inefficiente per KB grandi).

**Definizione 4.3** (Sistema di inferenza). Un **sistema di inferenza** (o sistema Hilbertiano) in una logica  $\mathcal{L}$  è composto da:

- Un insieme di assiomi  $\mathcal{A}$  (schemi di formule dati per buoni).
- Un insieme di regole di inferenza  $\mathcal{R} = \{R_1, \dots, R_n\}$  che permettono di derivare nuove formule a partire da formule note.

Una **derivazione** in tale sistema è una sequenza finita di formule ottenute applicando assiomi e regole del sistema.

**Definizione 4.4** (Correttezza e completezza). Dati una KB e un sistema di inferenza, si dice che un sistema di inferenza è:

- **corretto** se ogni formula derivata è conseguenza logica della KB.
- **completo** se ogni formula che è conseguenza logica della KB può essere derivata tramite le regole del sistema. Vedremo più avanti il concetto di derivazione.

**Tecniche di verifica:** come verifichiamo che  $\alpha \models \beta$ ?

- Costruisco una tavola di verità per  $\alpha \rightarrow \beta$  e verifico se è una tautologia.
- Utilizzo definizione di modello
- Dimostro  $\beta$  partendo da  $\alpha$  attraverso una serie di regole di inferenza corrette.

**Esempio 4.3** (Cavalieri e furfanti). Siano  $A, B$  abitanti di un'isola dove cavalieri dicono sempre la verità e furfanti mentono sempre.  $A$  afferma: "Io sono un furfante oppure  $B$  è cavaliere".

Formalizzazione:

- (1)  $A \rightarrow (\neg A \vee B)$
- (2)  $\neg A \rightarrow \neg(\neg A \vee B)$

| $(A$ | $\rightarrow$ | $(\neg A$ | $\vee$ | $B))$ | $\wedge$ | $(\neg A$ | $\rightarrow$ | $\neg$ | $(\neg A$ | $\vee$ | $B))$ |
|------|---------------|-----------|--------|-------|----------|-----------|---------------|--------|-----------|--------|-------|
| 1    | 1             | 0         | 1      | 1     | 1        | 0         | 1             | 0      | 0         | 1      | 1     |
| 1    | 0             | 0         | 0      | 0     | 0        | 0         | 1             | 1      | 0         | 0      | 0     |
| 0    | 1             | 1         | 1      | 1     | 0        | 1         | 0             | 0      | 1         | 1      | 1     |
| 0    | 1             | 1         | 1      | 0     | 0        | 1         | 0             | 0      | 1         | 1      | 0     |

Con tavola di verità si deduce l'unico modello con  $A = 1, B = 1$ . Quindi entrambi sono cavalieri.

**Dimostrazione diretta (tramite modelli):**

Dalla scomposizione logica si verifica che se  $\mathcal{M}$  soddisfa (1) e (2), allora  $\mathcal{M} \models A$  e  $\mathcal{M} \models B$ .

## 5 Alberi di Beth

Alla base della costruzione di questi alberi vi sono i controesempi: assegnazioni di verità che "falsifica" la formula (si dice che è un **contromodello** della formula).

**Osservazione 5.1.**  $A$  è una tautologia se e solo se  $A$  non ha controesempi

Il metodo di Beth è un algoritmo di **deduzione automatica** che si basa sulla ricerca di un contromodello per la formula  $A$ . In particolare

- Si suppone che esista un contromodello per  $A$
- Si costruisce un opportuno albero secondo regole non ambigue
- Costruzione termina dopo un numero finito di passi: se ho una contraddizione allora la formula di partenza non ammette un contromodello, altrimenti posso estrarre un controesempio (nessuna formula del ramo può essere ulteriormente sviluppata).

Ricordiamo che un albero binario è una struttura dati in cui ogni nodo ha al più due figli. Gli alberi semantici che costruiremo avranno **una formula** (o più formule) associata a ogni nodo. Un ramo è una sequenza di nodi connessi da archi che da una foglia arriva alla radice.

**Osservazione 5.2.** Per verificare tautologicità di una formula si parte dalla negazione di una formula. I passi di costruzione dei rami dell'albero sono guidati dalla semantica dei connettivi.

**Regole di costruzione:** Data una formula  $X \cdot Y$  con  $\cdot$  un connettivo, definiamo due regole di sviluppo dell'albero che corrispondono ai seguenti casi:

- La formula è vera
- La formula è falsa

|              | Formula vera                                                                           | Formula negata                                                                               |
|--------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| congiunzione | $\begin{array}{c} X \wedge Y \\   \\ X, Y \end{array}$                                 | $\begin{array}{c} \neg(X \wedge Y) \\ / \quad \backslash \\ \neg X \quad \neg Y \end{array}$ |
| disgiunzione | $\begin{array}{c} X \vee Y \\ / \quad \backslash \\ X \quad Y \end{array}$             | $\begin{array}{c} \neg(X \vee Y) \\   \\ \neg X, \neg Y \end{array}$                         |
| condizionale | $\begin{array}{c} X \rightarrow Y \\ / \quad \backslash \\ \neg X \quad Y \end{array}$ | $\begin{array}{c} \neg(X \rightarrow Y) \\   \\ X, \neg Y \end{array}$                       |
| negazione    |                                                                                        | $\begin{array}{c} \neg\neg X \\   \\ X \end{array}$                                          |

Si dice che:

- l'albero si **chiude** se **tutti** i rami si chiudono (ogni ramo presenta una contraddizione)
- l'albero è **aperto** se almeno un ramo rimane aperto (almeno un controesempio)

Sviluppo di un albero di Beth avviene attraverso una serie di stadi intermedi, dove ad ogni stadio si costruisce un albero di formule  $(\tau_1, \dots, \tau_n)$ .

- **Inizializzazione:**  $\tau_1$  inizializzato come un solo nodo etichettato con la **negazione** della formula di partenza.
- Al generico passo  $k$  si costruisce  $\tau_k$ , per cui si può verificare o che tutti i rami sono chiusi (fine costruzione e ogni ramo presenta una contraddizione) o esiste almeno un ramo non chiuso. In quest'ultimo caso si esamina il ramo e si verifica se ci sono formule sviluppabili
  - (a) Non ci sono formule sviluppabili: albero completato e aperto, per cui esiste un controesempio
  - (b) Ci sono formule sviluppabili, per cui si prolunga l'albero selezionando la formula sviluppabile e applicando le regole in tabella ??:
    - Nei casi  $(\wedge, \neg\vee, \neg\rightarrow, \neg\neg)$  si aggiunge esattamente **un figlio** alla foglia di ciascun ramo aperto che passa per il nodo esaminato
    - Nei casi  $(\neg\wedge, \vee, \rightarrow)$  si aggiungono esattamente **due figli** alla foglia di ciascun ramo aperto che passa per il nodo esaminato

**Osservazione 5.3.** Alcune osservazioni:

- Costruzione dell'albero di Beth termina sempre in un numero finito di passi
- L'albero soddisfa una delle seguenti proprietà:
  - **chiuso**
  - **completamente aperto**
- L'albero di Beth è chiuso se e solo se la formula di partenza è una tautologia
- Il metodo è corretto e completo

## 6 Sistema Hilbertiano

Un sistema di inferenza è un insieme di **assiomi** (schemi di formule dati per buoni). Assieme agli assiomi, si definiscono delle **regole di inferenza**  $\mathcal{R} = \{R_1, \dots, R_n\}$  che permettono di derivare nuove formule a partire da formule note.



**Definizione 6.1** (Sistema Hilbertiano). Un sistema di inferenza Hilbertiano (HAL) è un sistema di inferenza definito nella logica proposizionale. Si utilizza il linguaggio della logica proposizionale definito tramite i soli connettivi  $\neg, \rightarrow$ . Il sistema di inferenza Hilbertiano è così definito:

- Assiomi:
  - $A \rightarrow (B \rightarrow A)$
  - $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$
  - $(\neg A \rightarrow \neg B) \rightarrow (B \rightarrow A)$
- Regole di inferenza: Modus ponens
 
$$\frac{A, A \rightarrow B}{B}$$

**Definizione 6.2** (Derivazione in HAL). Sia  $\Phi$  un insieme di formule della logica proposizionale. Sia  $A$  una formula. Una sequenza **finita e non vuota** di formule  $A_1, \dots, A_n \in \Phi$  si dice **derivazione** di  $A$  da  $\Phi$  se e solo se valgono le seguenti proprietà:

- $A_n = A$
- Ogni formula  $A_i$  soddisfa una tra le seguenti condizioni:
  - è un assioma
  - $A_i \in \Phi$  (formula nella base di conoscenza)
  - $A_i$  è ottenuta tramite l'applicazione del modus ponens da due formule  $A_j, A_k$  con  $j, k < i$

Se esiste una derivazione di  $A$  da  $\Phi$ , si dice che  $A$  è **derivabile** da  $\Phi$  (oppure  $A$  è conseguenza sintattica di  $\Phi$ ). Si scrive:

$$\Phi \vdash A \quad \text{oppure} \quad \Phi \vdash_{\text{HAL}} A$$

**Osservazione 6.1.** *Derivabilità e conseguenza logica non sono equivalenti.*

**Definizione 6.3** (Dimostrazione teorema). 1. Una dimostrazione di una formula  $A$  è una sequenza di formule tali che ognuna è un assioma oppure segue dalle formule che la precedono per le regole di inferenza.

2. Se esiste una dimostrazione di  $A$  si dice che  $A$  è **dimostrabile** e si scrive  $\vdash A$ .

**Definizione 6.4** (Proprietà di  $\vdash$ ). Siano  $A, B$  formule di  $\mathcal{L}_{\text{prop}}$  e  $\Phi, \Gamma$  insiemi di formule di  $\mathcal{L}_{\text{prop}}$ . Allora:

1.  $A \vdash A$
2. Se  $\Phi \vdash A$ , allora  $\Phi \cup \Gamma \vdash A$
3. Se  $\Phi \vdash A$  e  $A \vdash B$ , allora  $\Phi \vdash B$
4. Se  $\vdash A$  allora  $\Gamma \vdash A$
5. Se  $\Phi \vdash A$  allora esiste un sottoinsieme finito  $\Phi' \subseteq \Phi$  tale che  $\Phi' \vdash A$  (compattezza)
6. Se  $\Phi \subseteq \Gamma$  e  $\Phi \vdash A$ , allora  $\Gamma \vdash A$  (monotonicità)

**Definizione 6.5** (Teorema di deduzione (HAL)). Dato un insieme di formule  $\Phi$ ,  $\Phi \cup \{A\} \vdash B$  se e solo se  $\Phi \vdash A \rightarrow B$ . Ovvero  $B$  è derivabile da  $\Phi \cup \{A\}$  se e solo se  $A \rightarrow B$  è derivabile da  $\Phi$ .

$$A \vdash B \iff \vdash A \rightarrow B$$

Ovvero  $B$  è derivabile da  $A$  se e solo se  $A \rightarrow B$  è un teorema.

**Osservazione 6.2.** *Teorema di deduzione in HAL stabilisce una condizione di derivabilità di una formula. Il teorema di deduzione che abbiamo visto per la conseguenza logica invece stabilisce il legame tra tautologia e conseguenza logica.*

*Dimostrazione.* ( $\leftarrow$ ) supponiamo che  $\Phi \vdash A \rightarrow B$ . Allora esiste una derivazione  $S := C_1, \dots, C_n$  tale che  $C_n = A \rightarrow B$ . Se consideriamo l'insieme esteso di formule  $\Phi \cup \{A\}$  posso definire la sequenza  $S' := C_1, \dots, C_{n-1}, A, C_n, C_{n+1} = A$ . Allora dal Modus ponens applicato a  $C_n = A \rightarrow B$  e  $C_{n+1} = A$  segue  $B$ . Quindi  $\Phi \cup \{A\} \vdash B$ .

( $\rightarrow$ ) Sia  $S := C_1, \dots, C_n$  una derivazione di  $B$  da  $\Phi \cup \{A\}$ . In particolare  $C_n = B$ . Se per ogni  $k$  dimostriamo che  $\Phi \vdash A \rightarrow C_k$ , allora avremmo in particolare  $\Phi \vdash A \rightarrow B$ . Procediamo per induzione su  $k$ .

1. **Caso base:** supponiamo  $C_k$  derivabile in un solo passo. Dunque o è un assioma o una formula di  $\Phi \cup \{A\}$ .

(a) Se  $C_k$  è un assioma allora possiamo trovare la seguente derivazione:

- i.  $C_k$  è un assioma
- ii.  $C_k \rightarrow (A \rightarrow C_k)$  (Assioma 1)
- iii.  $A \rightarrow C_k$  (Modus ponens)

dunque per ogni  $k$ , se  $C_k$  è un assioma allora  $\Phi \vdash A \rightarrow C_k$ .

- (b)  $C_k$  è una formula di  $\Phi \cup \{A\}$  dunque  $C_k \in \Phi$  oppure  $C_k \in \{A\}$ .

- i. Se  $C_k \in \Phi$  allora considero la derivazione precedente.
- ii. Se  $C_k \in \{A\}$  allora  $C_k = A$ . Dunque  $A \rightarrow C_k$  diventa  $A \rightarrow A$  (e sappiamo che  $\vdash A \rightarrow A$ )

- (c)  $C_k$  è ottenuta da due formule precedenti mediante Modus ponens. Siano  $i, j < k$  tali che  $C_j = C_i \rightarrow C_k$ . Per ipotesi induttiva abbiamo che  $\Phi \vdash A \rightarrow C_i$  e  $\Phi \vdash A \rightarrow C_j$  e mostriamo che  $\Phi \vdash A \rightarrow C_k$ . Costruiamo il seguente argomento:

- i.  $A \rightarrow C_i$  (per ipotesi)
- ii.  $A \rightarrow (C_i \rightarrow C_k)$  (per ipotesi  $C_j = C_i \rightarrow C_k$ )
- iii.  $A \rightarrow (C_i \rightarrow C_k) \rightarrow ((A \rightarrow C_i) \rightarrow (A \rightarrow C_k))$  (Assioma 2)
- iv.  $(A \rightarrow C_i) \rightarrow (A \rightarrow C_k)$  (Modus ponens)
- v.  $A \rightarrow C_k$  (Modus ponens)

□

**Definizione 6.6** (Proprietà del sistema Hilbertiano). Il sistema Hilbertiano è **corretto** e **completo** rispetto alla semantica della logica proposizionale.

- **Correttezza:** Se  $\vdash_{\text{HAL}} A$ , allora  $\models A$  (ogni formula derivabile in HAL è una tautologia)
- **Completezza:** Se  $\models A$ , allora  $\vdash_{\text{HAL}} A$  (se  $A$  è una formula valida allora è derivabile in HAL)

Dimostriamo la correttezza del sistema Hilbertiano.

$\vdash_{\text{HAL}} \alpha \implies \models \alpha$ . Dall'assioma 1:

$$A \rightarrow (B \rightarrow A)$$

e si mostra che gli assiomi 1,2 e 3 sono tautologie (con definizione di modello o tavole di verità). Si dimostra poi la conoscenza della regola di inferenza Modus ponens. □

**Osservazione 6.3.** Si parla di *ragionamento ritrattabile* per esprimere il fatto che l'aggiunta di informazione a una base di conoscenza può portare alla ritrattazione di formule che precedentemente potevano essere derivate dal sistema.

Una base di conoscenza si dice consistente se esiste almeno un modello che la soddisfa.

**Definizione 6.7** (Letterale). Un **letterale** in  $\mathcal{L}_{\text{prop}}$  è una formula atomica o la sua negazione.

**Definizione 6.8** (Clausola). Una **clausola** è una disgiunzione di letterali. Si dice unitaria se contiene esattamente un letterale.

Si può rappresentare una clausola elencando l'insieme dei letterali che la compongono.

$$C := l_1 \vee \dots \vee l_n \longrightarrow \{l_1, \dots, l_n\}$$

Osserviamo che l'aggiunta di uno o più letterali a una clausola si può rappresentare come unione di insiemi.

**Definizione 6.9** (Forma normale congiuntiva (CNF)). Una formula in  $\mathcal{L}_{\text{prop}}$  espressa come congiunzione di clausole si dice in **forma normale congiuntiva**.

**Definizione 6.10** (Equivalenza logica). Due formule della logica proposizionale  $\alpha, \beta$  si dicono **logicamente equivalenti** ( $\alpha \equiv \beta$ ) se e solo se  $\alpha \models \beta$  e  $\beta \models \alpha$ .

**Osservazione 6.4.** Dal **teorema di deduzione** deriviamo che  $\alpha \equiv \beta \iff \alpha \leftrightarrow \beta$  è una tautologia.

Degli esempi sono:

|                                                                                                        |                                      |
|--------------------------------------------------------------------------------------------------------|--------------------------------------|
| $(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$                                                   | commutatività di $\wedge$            |
| $(\alpha \vee \beta) \equiv (\beta \vee \alpha)$                                                       | commutatività di $\vee$              |
| $((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$                   | associatività di $\wedge$            |
| $((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$                           | associatività di $\vee$              |
| $\neg(\neg\alpha) \equiv \alpha$                                                                       | eliminazione della doppia negazione  |
| $(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$                                 | contrapposizione                     |
| $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$                                            | eliminazione dell'implicazione       |
| $(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$ | eliminazione del bicondizionale      |
| $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$                                         | De Morgan                            |
| $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$                                         | De Morgan                            |
| $(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$       | distributività di $\wedge$ su $\vee$ |
| $(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$         | distributività di $\vee$ su $\wedge$ |

## 6.1 Trasformazione in CNF

Ogni formula della logica proposizionale è logicamente equivalente a una **congiunzione di clausole**.

Per la trasformazione in CNF si può definire un algoritmo di **conversione**, che consiste nell'applicazione delle seguenti regole:

- $A \rightarrow B$  è rimpiazzato dalla forma equivalente  $\neg A \vee B$ .
- $A \leftrightarrow B$  è rimpiazzato dalla forma equivalente  $(\neg A \vee B) \wedge (A \vee \neg B)$ .
- $\neg(A \wedge B)$  è rimpiazzato dalla forma equivalente  $\neg A \vee \neg B$  (De Morgan).
- $\neg(A \vee B)$  è rimpiazzato dalla forma equivalente  $\neg A \wedge \neg B$  (De Morgan).
- $\neg(\neg A)$  è rimpiazzato dalla forma equivalente  $A$  (eliminazione della doppia negazione).
- Applicazione delle proprietà distributive:

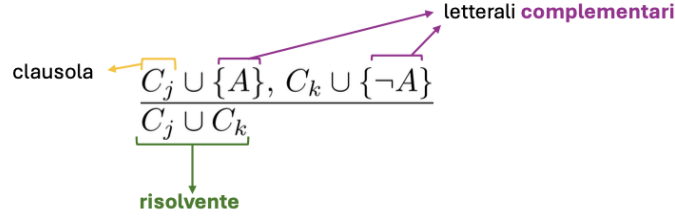
$$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C); \quad A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C).$$

- Applicazione delle proprietà associative:

$$A \wedge (B \wedge C) \equiv (A \wedge B) \wedge C; \quad A \vee (B \vee C) \equiv (A \vee B) \vee C.$$

## 6.2 Sistema di inferenza basato su risoluzione (RES)

Si tratta di un sistema di inferenza che si basa su un'unica regola di inferenza. Il linguaggio della logica proposizionale che si utilizza è quella della **CNF**. La regola di inferenza si applica alle clausole. Non ci sono assiomi. La regola di inferenza (risoluzione) si esprime come:



La regola di risoluzione unitaria (clausola con un letterale)  
Sia  $C := l_1 \vee \dots \vee l_n$  e sia  $l$  un letterale tale che  $l = ¬l_i$  con  $i$  fissato compreso tra 1 e  $n$ .

$$\frac{C, l}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_n}$$

Per esteso:

$$\frac{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_n, ¬l_i}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_n}$$

**Definizione (Derivazione in RES).** Sia  $\Phi$  un insieme di clausole e  $A$  una formula (tipicamente una clausola). Una **derivazione** di  $A$  da  $\Phi$  tramite risoluzione è una sequenza finita di clausole  $C_1, \dots, C_n$  tale che  $C_n = A$  e per ogni  $i \in \{1, \dots, n\}$ :

- $C_i \in \Phi$ , oppure
- $C_i$  è il **risolvente** di una coppia di clausole  $C_j, C_k$  con  $j, k < i$  (cioè,  $C_i$  è ottenuto applicando la regola di risoluzione a due clausole precedenti nella sequenza).

Se esiste una derivazione di  $A$  da  $\Phi$  per risoluzione, si dice che  $A$  è **derivabile** da  $\Phi$  per/tramite risoluzione e si scrive:

$$\Phi \vdash_{RES} A$$

Se  $\Phi = \emptyset$ , si dice che  $A$  è un **teorema** (nel sistema RES), o che  $A$  è **dimostrabile** per risoluzione, e si scrive  $\vdash_{RES} A$ .

### Esempio 1: Derivazione per risoluzione

Sia  $\Phi = \{¬p \vee q, p \vee r, ¬q \vee s\}$ . Vogliamo mostrare che  $\Phi \vdash_{RES} r \vee s$ . La derivazione può essere scritta come segue:

1.  $¬p \vee q$  (Premessa da  $\Phi$ )
2.  $p \vee r$  (Premessa da  $\Phi$ )
3.  $¬q \vee s$  (Premessa da  $\Phi$ )
4.  $q \vee r$  (Risolvente di 1 e 2, eliminando  $p$  e  $¬p$ )
5.  $r \vee s$  (Risolvente di 4 e 3, eliminando  $q$  e  $¬q$ )

Quindi, abbiamo dimostrato che  $r \vee s$  è derivabile da  $\Phi$ . Possiamo anche usare la notazione insiemistica per le clausole:

1.  $\{¬p, q\}$
2.  $\{p, r\}$
3.  $\{¬q, s\}$

4.  $\frac{\{\neg p, q\}, \{p, r\}}{\{q, r\}}$  (Risoluzione su  $p$ )
5.  $\frac{\{q, r\}, \{\neg q, s\}}{\{r, s\}}$  (Risoluzione su  $q$ )

### Esempio 2: Derivazione della clausola vuota

Sia  $\Phi = \{p \vee q, \neg p, \neg q\}$ . Vogliamo vedere cosa si può derivare.

1.  $p \vee q$  (Premessa da  $\Phi$ )
2.  $\neg p$  (Premessa da  $\Phi$ )
3.  $\neg q$  (Premessa da  $\Phi$ )
4.  $q$  (Risolvente di 1 e 2)
5.  $\perp$  (Risolvente di 4 e 3)

Abbiamo derivato la clausola vuota, indicata con  $\perp$ . La clausola vuota rappresenta la contraddizione (Falso). Se  $\Phi \vdash_{RES} \perp$ , significa che l'insieme di clausole  $\Phi$  è **insoddisfacibile**. In questo caso, l'insieme  $\{p \vee q, \neg p, \neg q\}$  è insoddisfacibile.

### Correttezza della Regola di Risoluzione

La regola di risoluzione è corretta perché, se due clausole

$$C_1 = A \vee l \quad \text{e} \quad C_2 = B \vee \neg l$$

sono vere in una certa interpretazione  $I$ , allora anche il loro risolvente

$$C = (A \vee l) \text{ res } (B \vee \neg l) = A \vee B$$

risulta vero in  $I$ . In dettaglio:

- Se  $l$  è vero in  $I$ , allora  $\neg l$  è falso; poiché  $C_2$  è vera, almeno un letterale di  $B$  deve essere vero, quindi  $A \vee B$  è vero in  $I$ .
- Se  $l$  è falso in  $I$ , allora  $\neg l$  è vero; poiché  $C_1$  è vera, almeno un letterale di  $A$  deve essere vero, quindi  $A \vee B$  è vero in  $I$ .

Poiché ogni risolvente preserva la verità delle clausole di partenza, *ogni singola applicazione* della regola di risoluzione è corretta.

### Proprietà della Risoluzione

Il sistema di inferenza RES gode di due proprietà fondamentali: correttezza e completezza.

- **Correttezza (Soundness):** Se una formula  $A$  è derivabile da un insieme di formule  $\Phi$  usando la risoluzione, allora  $A$  è una conseguenza logica di  $\Phi$ . Formalmente:

$$\text{Se } \Phi \vdash_{RES} A, \text{ allora } \Phi \models A$$

*Dimostrazione (cenno):* Si basa sul fatto che la regola di risoluzione (v. sezione precedente) preserva la verità. Se le clausole genitrici  $C_1 = A \vee l$  e  $C_2 = B \vee \neg l$  sono vere in un'interpretazione  $I$ , allora anche il loro risolvente  $C = A \vee B$  è vero in  $I$ . Poiché ogni passo della derivazione produce una clausola che è conseguenza logica delle premesse, la clausola finale  $A$  sarà anch'essa conseguenza logica di  $\Phi$ . In particolare, se  $\Phi \vdash_{RES} \perp$ , allora  $\Phi \models \perp$ , il che significa che  $\Phi$  è insoddisfacibile.

- **Completezza (Completeness):** Se una formula  $A$  è una conseguenza logica di un insieme di formule  $\Phi$  (in logica proposizionale), allora  $A$  è derivabile da  $\Phi$  usando la risoluzione. Formalmente:

$$\text{Se } \Phi \models A, \text{ allora } \Phi \vdash_{RES} A$$

La dimostrazione della completezza della risoluzione è più complessa e spesso viene dimostrata attraverso la **completezza per refutazione**.

### Risoluzione e Refutazione

La risoluzione è particolarmente potente quando usata come **metodo di refutazione**. L'idea si basa sulla seguente equivalenza fondamentale:

$$\Phi \models A \iff \Phi \cup \{\neg A\} \text{ è insoddisfacibile}$$

Ricordando che un insieme di formule è insoddisfacibile se e solo se da esso si può derivare la clausola vuota  $\perp$  tramite risoluzione (grazie alla correttezza e completezza per refutazione), possiamo stabilire un metodo per dimostrare  $\Phi \models A$ :

$$\Phi \models A \iff \Phi \cup \{\neg A\} \vdash_{RES} \perp$$

### Procedura di Dimostrazione per Refutazione basata su Risoluzione

Per dimostrare che  $\Phi \models A$  usando la risoluzione:

1. **Converti in CNF:** Trasforma tutte le formule nell'insieme  $\Phi \cup \{\neg A\}$  in forma normale congiuntiva (CNF). Sia  $\Phi'$  l'insieme di tutte le clausole ottenute.
2. **Applica Risoluzione:** Utilizza ripetutamente la regola di risoluzione sull'insieme di clausole  $\Phi'$  fino a quando non si verifica una delle seguenti condizioni:
  - Si deriva la clausola vuota ( $\perp$ ). In questo caso,  $\Phi \cup \{\neg A\}$  è insoddisfacibile, e quindi la dimostrazione  $\Phi \models A$  ha successo.
  - Non è possibile applicare ulteriormente la regola di risoluzione per generare nuove clausole, e la clausola vuota non è stata derivata. In questo caso, l'insieme  $\Phi \cup \{\neg A\}$  è soddisfacibile, e quindi  $\Phi \not\models A$ .

Questo metodo è garantito funzionare perché la risoluzione (con l'aggiunta eventuale della fattorizzazione, vedi sotto) è completa per refutazione per la logica proposizionale.

### Esempio di Refutazione

Vogliamo dimostrare che  $\{p \rightarrow q, q \rightarrow r\} \models p \rightarrow r$ .

1. **Formulo l'insieme  $\Phi \cup \{\neg A\}$ :**

$$\Phi = \{p \rightarrow q, q \rightarrow r\}$$

$$A = p \rightarrow r$$

$$\neg A = \neg(p \rightarrow r) \equiv \neg(\neg p \vee r) \equiv p \wedge \neg r$$

L'insieme da refutare è  $\{p \rightarrow q, q \rightarrow r, p \wedge \neg r\}$ .
2. **Converto in CNF:**

$$p \rightarrow q \equiv \neg p \vee q \text{ (Clausola: } \{\neg p, q\})$$

$$q \rightarrow r \equiv \neg q \vee r \text{ (Clausola: } \{\neg q, r\})$$

$$p \wedge \neg r \text{ (Si spezza in due clausole: } p \text{ e } \neg r)$$

Clausole:  $p \equiv \{p\}$ ,  $\neg r \equiv \{\neg r\}$

L'insieme di clausole  $\Phi'$  è:  $\{\{\neg p, q\}, \{\neg q, r\}, \{p\}, \{\neg r\}\}$ .
3. **Applico Risoluzione:**
  - (a)  $\{\neg p, q\}$  (da  $\Phi'$ )
  - (b)  $\{\neg q, r\}$  (da  $\Phi'$ )
  - (c)  $\{p\}$  (da  $\Phi'$ )
  - (d)  $\{\neg r\}$  (da  $\Phi'$ )
  - (e)  $\{q\}$  (Risolvente di 1 e 3)
  - (f)  $\{r\}$  (Risolvente di 2 e 5)
  - (g)  $\perp$  (Risolvente di 6 e 4)

Poiché abbiamo derivato la clausola vuota  $\perp$ , abbiamo dimostrato che  $\{p \rightarrow q, q \rightarrow r\} \models p \rightarrow r$ .

### Fattorizzazione

A volte, durante l'applicazione della risoluzione, una clausola può contenere più copie dello stesso letterale. Ad esempio, applicando la risoluzione a  $p \vee q$  e  $p \vee \neg q$  rispetto a  $q$ , otterremmo  $p \vee p$ . Formalmente, il risolvente di  $C_1 = A \vee l$  e  $C_2 = B \vee \neg l$  è  $(A \setminus \{\neg l\}) \cup (B \setminus \{l\})$ . Se  $A$  o  $B$  contengono  $l$  o  $\neg l$ , questi non vengono rimossi.

La regola di **fattorizzazione** permette di semplificare le clausole eliminando i letterali duplicati:

$$\frac{C \vee l \vee l}{C \vee l}$$

Questa regola può essere applicata a qualsiasi clausola in qualsiasi momento. Spesso, viene implicitamente integrata nella definizione della regola di risoluzione assumendo che le clausole siano insiemi di letterali, oppure viene applicata come passo di semplificazione dopo la risoluzione. Adottiamo la convenzione che una clausola è un insieme di letterali, quindi la duplicazione è automaticamente gestita:  $l \vee l \equiv l$ . Nell'esempio  $\frac{p \vee q, p \vee \neg q}{p \vee p}$ , applicando questa convenzione (o la fattorizzazione), il risultato è semplicemente  $p$ .

La risoluzione, combinata con la fattorizzazione (se non si usa la convenzione insiemistica), fornisce un metodo di refutazione completo per la logica proposizionale. Questo significa che se un insieme di clausole è insoddisfacibile, allora esiste una derivazione della clausola vuota  $\square$  utilizzando risoluzione e fattorizzazione.

## 6.3 Completezza della Risoluzione

**Teorema 6.1** (Completezza della Risoluzione come metodo di refutazione). *Enunciamo le due formulazioni equivalenti del teorema:*

*Formulazione 1: Sia  $\Phi$  un insieme di formule della logica proposizionale. Sia  $A$  una formula. Se  $\Phi \models A$ , allora esiste una derivazione per risoluzione di  $\square$  da  $\Phi \cup \{\neg A\}$ . In simboli:*

$$\text{Se } \Phi \models A, \text{ allora } \Phi \cup \{\neg A\} \vdash_{RES} \square$$

*Formulazione 2: Se un insieme di clausole  $\Phi$  è insoddisfacibile, allora esiste  $\Phi \vdash_{RES} \square$ .*

**Corollario 6.1.** *Se  $\Phi$  è un insieme di clausole, vale che  $\Phi$  è insoddisfacibile se e solo se  $\Phi \vdash_{RES} \square$ .*

Useremo in prevalenza la formulazione 2, che ricordiamo essere equivalente alla prima.

*Dimostrazione.* Definiamo la chiusura di  $\Phi$ ,  $RC(\Phi)$ , come l'insieme di tutte le clausole ottenute da  $\Phi$  per risoluzione.

Procediamo per assurdo dimostrando che se  $\Phi \not\vdash_{RES} \square$ , allora  $\Phi$  è soddisfacibile.

Osserviamo che se  $\Phi \not\vdash_{RES} \square$ , allora  $\square \notin RC(\Phi)$ .

Per dimostrare che  $\Phi$  è soddisfacibile, costruiamo un modello di  $\Phi$ .

Siano  $P_1, \dots, P_k$  le variabili proposizionali che compaiono in  $\Phi$ . Occorre trovare una assegnazione di valori di verità alle variabili in modo che tutte le formule in  $\Phi$  siano vere.

**Nota:** se  $\square \in \Phi$ , allora esisterebbe almeno una clausola falsa, ma per ipotesi  $\square \notin RC(\Phi)$ .

Definiamo il seguente modello  $\mathcal{M}$  assegnando i seguenti valori di verità alle variabili per  $i = 1, \dots, k$ :

- a)  $\mathcal{M}$  assegna **0 (falso)** a  $P_i$  se esiste una clausola  $\bar{C} \in RC(\Phi)$  tale che  $\neg P_i \subseteq \bar{C}$  e **tutti gli altri letterali di  $\bar{C}$  sono falsi nell'assegnamento scelto per  $P_1, \dots, P_{i-1}$**
- b)  $\mathcal{M}$  assegna **1 (vero)** a  $P_i$  altrimenti

**Mostriamo che:**  $\mathcal{M}$  è un modello di  $\Phi$ .

*Dimostrazione.* Supponiamo per assurdo che  $\mathcal{M} \not\models \Phi$ . Allora esiste una assegnazione di valori di verità ai letterali tale che per qualche indice  $i$ , il valore di verità di  $P_i$  è 0 e per ogni  $C \in \Phi$ .

Sia dunque  $i$  il primo indice per cui il valore di verità di  $P_i$  falsifica una clausola.

Una clausola è falsa se sono falsi tutti i suoi letterali, dunque affinché  $P_i$  falsifichi  $C$  è necessario che tutti i letterali della clausola siano già stati falsificati per le assegnazioni di  $P_1, \dots, P_{i-1}$ .

Dunque  $C$  p della forma

1.  $C = (\text{false} \vee \text{false} \vee \dots \vee P_i)$  oppure
2.  $C = (\text{false} \vee \text{false} \vee \dots \vee \neg P_i)$

Se una sola di queste clausole è in  $RC(\Phi)$ , allora il valore di verità di  $P_i$  rende vera la clausola.

Infatti se  $P_i = 0$ , per come è definito il modello,  $P_i$  rende vera la clausola della forma  $C_2$ . Se invece  $P_i = 1$ , allora  $P_i$  rende vera la clausola della forma  $C_1$ .

Dunque una clausola di questa forma può essere falsificata dall'assegnamento del valore di verità di  $P_i$  solo se sia  $C_1$  che  $C_2$  sono in  $RC(\Phi)$ .

In questa caso  $RC(\Phi)$  **contiene il risolvente** e di  $C_1$  e  $C_2$  per definizione.

Il risolvente delle due clausole avrà tutti i letterali già falsificati dalle assegnazioni di  $P_1, \dots, P_{i-1}$ .

Dunque il risolvente è falso.

Si ha dunque una **contraddizione** dato che si era supposta che la prima falsificazione avvenisse al passo  $i$ , mentre invece si è trovata una falsificazione al passo  $i - 1$ . Dunque  $\mathcal{M} \models \Phi$ . E concludiamo che se  $\Phi$  non contiene  $\square$ , allora  $\Phi$  è soddisfacibile, dunque la tesi.  $\square$

$\square$

## 6.4 Un algoritmo per la Risoluzione

Una prima soluzione, chiaramente non ottimale, è quella di effettuare tutte le possibili risoluzioni a partire dalla base di conoscenza (KB) fino a quando non si trova la clausola vuota.

Date una **base di conoscenza** KB e una formula della logica proposizionale  $a$ , l'algoritmo restituisce l'insieme di tutte le possibili clausole derivabili per risoluzione. Se tra queste vi è la clausola vuota si conclude che  $a$  deriva da KB.

---

**Algorithm 1** PL-RISOLUZIONE( $KB, \alpha$ ) returns **true** oppure **false**

---

**Require:**  $KB$ , la base di conoscenza;  $\alpha$ , formula della logica proposizionale

```

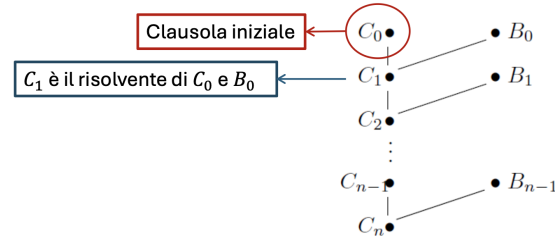
1:  $clausole \leftarrow$  l'insieme di clausole nella rappresentazione CNF di  $KB \wedge \neg\alpha$ 
2:  $nuove \leftarrow \{\}$ 
3: loop
4:   for each coppia di clausole  $C_i, C_j$  in  $clausole$  do
5:      $risolventi \leftarrow PL-RISOLVI(C_i, C_j)$ 
6:     if  $risolventi$  contiene la clausola vuota then
7:       return true
8:     end if
9:      $nuove \leftarrow nuove \cup risolventi$ 
10:  end for
11:  if  $nuove \subseteq clausole$  then
12:    return false
13:  end if
14:   $clausole \leftarrow clausole \cup nuove$ 
15: end loop
```

---

Esista un algoritmo più efficiente per la risoluzione, detto di **risoluzione lineare**.

1. Si seleziona una clausola  $C_0$  dalla base di conoscenza  $\Phi$ .
2. Si risolve  $C_0$  con una clausola di  $\Phi$  dove compare un letterale complementare, oppure con una clausola precedentemente derivata.





Nelle  $B_0, B_1, \dots$  posso avere anche delle clausole derivate. Quindi una volta che si è derivata una clausola usiamo il risolvente come nuova clausola di base.

$$\begin{array}{c}
 \frac{P \vee Q \quad \neg P \vee Q}{Q} \quad \frac{P \vee \neg Q}{P} \quad \frac{\neg P \vee \neg Q}{\neg Q} \quad Q \\
 \hline
 \square
 \end{array}$$

**Osservazione 6.5.** La risoluzione lineare permette di evitare la ridondanza che caratterizza PL risoluzione, ma è cruciale la scelta della clausola iniziale. Infatti la completezza del sistema RES non garantisce che esista una refutazione lineare indipendentemente dalla scelta di  $C_0$  come clausola di partenza.

In generale si tiene presente il seguente risultato: se  $\Phi = \Phi' \cup C$  è insoddisfacibile e con  $\Phi'$  soddisfacibile, allora esiste una refutazione lineare di  $\Phi$  con  $C$  come clausola di partenza.

## 6.5 Algoritmi di deduzione automatica per clausole definite

Quando la base di conoscenza è formata da tipi di clausole ben definiti, è possibile utilizzare degli algoritmi di inferenza ad hoc che hanno complessità lineare nel numero di clausole della base di conoscenza. Si distinguono:

- Concatenazione in avanti
- Concatenazione all'indietro

### 6.5.1 Clausole di Horn

**Definizione 6.11.** Un letterale si dice **negativo** se è della forma  $\neg p$  (negazione di una variabile proposizionale). Si dice **positivo** altrimenti.

**Definizione 6.12** (Clausole di Horn). Una clausola si dice **di Horn** se ha al più un letterale positivo.

**Definizione 6.13** (Clausole definite). Una clausola si dice **definita** se esattamente uno dei suoi letterali è positivo.

- Se costituita da un solo letterale positivo, si dicono **fatti**
- Se costituita da più letterali si dicono **regole**

**Definizione 6.14** (Clausole obiettivo). Una clausola si dice **obiettivo** se non contiene letterali positivi. Osserviamo che:

- La clausola vuota  $\square$  è una particolare clausola obiettivo
- Si parla di clausole obiettivo perché in un programma logico queste costituiscono i goal, l'obiettivo che si vuole dimostrare. In altre parole dato un goal  $G$ , l'esecuzione di un programma logico consiste nel dimostrare che  $KB \cup \{\neg G\}$  è insoddisfacibile.

### 6.5.2 Concatenazione in avanti per clausole definite

Sia  $KB$  una base di conoscenza formata da clausole definite (un solo letterale positivo). Sia data un'interrogazione  $q$  di  $KB$  (query). La concatenazione in avanti è un algoritmo di deduzione automatica che stabilisce se  $q$  segue logicamente da  $KB$ . L'algoritmo agisce come segue:

1. Se le premesse di una regola di  $KB$  sono vere, si aggiunge la conclusione all'insieme di  $KB$ . Si deduce che ogni inferenza è effettuata applicando il Modus Ponens.
2. L'algoritmo termina se  $q$  è aggiunta ai fatti noti o non è più possibile effettuare inferenze

L'algoritmo di concatenazione in avanti è corretto.

---

```

1: function PL-CA-CONSEGUE?( $KB, q$ ) returns true oppure false
2:   inputs:
3:      $KB$ , la base di conoscenza (un insieme di clausole proposizionali definite)
4:      $q$ , la query (un simbolo proposizionale)
5:      $conto \leftarrow$  una tabella, dove  $conto[c]$  è il numero di simboli della premessa della clausola  $c$ 
6:      $inferiti \leftarrow$  una tabella, dove  $inferiti[s]$  è inizialmente false per tutti i simboli
7:      $coda \leftarrow$  una coda di simboli, che contiene inizialmente quelli noti come veri nella  $KB$ 
8:
9:   while  $coda$  non è vuota do
10:     $p \leftarrow \text{POP}(coda)$ 
11:    if  $p = q$  then
12:      return true
13:    end if
14:    if  $inferiti[p] = \text{false}$  then
15:       $inferiti[p] \leftarrow \text{true}$ 
16:      for all clausola  $c$  in  $KB$  dove  $p$  è in  $c.PREMESSA$  do
17:        decrementa  $conto[c]$ 
18:        if  $conto[c] = 0$  then
19:          aggiungi  $c.CONCLUSIONE$  a  $coda$ 
20:        end if
21:      end for
22:    end if
23:  end while
24: return false

```

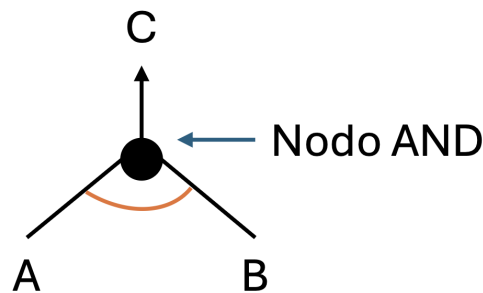
---

**Osservazione 6.6.** • *La concatenazione in avanti è un ragionamento guidato dai dati, dato che si ragiona a partire dai fatti noti nella base di conoscenza.*

- *L'algoritmo è corretto e completo. Ogni conseguenza logica di  $KB$  può essere dedotta*
- *La concatenazione in avanti può essere interpretata come una ricerca sul grafo AND-OR della base di conoscenza  $KB$ .*

Un grafo AND-OR è una rappresentazione sotto forma di grafo di una base di conoscenza formata da clausole definite. Inizialmente si aggiungono come nodi i fatti noti nella base di conoscenza, quindi si procede aggiungendo **nodi corrispondenti** a simboli inferiti per MPP in quanto conseguenti di regole della  $KB$ .

**Esempio 6.1.** Se  $A, B$  sono due nodi e nella  $KB$  c'è la regola  $A \wedge B \rightarrow C$ , allora si aggiunge un nodo simbolico AND come figlio di  $A$  e  $B$ .



### 6.5.3 Concatenazione all'indietro

L'algoritmo di concatenazione all'indietro parte **dalla query** e risale:

- Se la query è un fatto, l'algoritmo termina
- Se la query non è nota, si cercano le clausole che hanno la query come conclusione. Se le premesse sono dimostrabili essere vere allora la query segue.

In maniera simile alla concatenazione in avanti si può costruire un grafo AND-OR per la concatenazione all'indietro.

## 7 Logica del primo ordine

La conoscenza dichiarativa abbiamo visto che è una rappresentazione di oggetti, fatti e relazioni tra essi tramite asserzioni vere o false. Per codificare la conoscenza sul mondo da parte di un agente, e definire le regole del ragionamento che l'agente deve poter effettuare, si deve introdurre un opportuno linguaggio formale. Questo deve avere:

- Sintassi: stabilisce quali espressioni sono ben formate
- Semantica: attribuisce un valore di verità alle espressioni

Dato un linguaggio formale è possibile definire sistemi di inferenza.

### 7.1 Sintassi della logica del primo ordine

Gli elementi di base della logica del primo ordine rappresentano sempre oggetti o concetti di cui si specificano delle proprietà:

- **Costanti e variabili:** simboli che rappresentano oggetti di cui si parla
- **Predicati:** simboli che rappresentano proprietà degli oggetti

Predicati e costanti forniscono un modo per formare espressioni semplici. Espressioni più complesse si possono ottenere utilizzando i **quantificatori**:

- $\forall$  si legge "per ogni"
- $\exists$  si legge "esiste"

Vogliamo dunque definire come combinare simboli costanti, predicati, connettivi e quantificatori per formare espressioni ben formate, partendo dagli enunciati atomici e arrivando a definire formule ben formate del linguaggio.

Definiamo i simboli ammessi dalla logica del primo ordine:

- **Simboli logici:**
  - Variabili: insieme numerabile di variabili

- Connettivi logici e simboli proposizionali costanti
- Quantificatori
- **Simboli non logici:**
  - Simboli di predicato: insieme numerabile di simboli di predicato di arità  $n$
  - Costanti individuali
  - Simboli funzionali (funtori): insieme numerabile di funzioni di arità  $n$
- **Simboli separatori** (parentesi, virgole)

Noi assumiamo che l'insieme dei simboli di predicato contenga l'**uguaglianza** ( $=$ )

**Definizione 7.1** (Termini). Sono le unità sintattiche di base del linguaggio. Dato un linguaggio del primo ordine  $\mathcal{L}$ , si definisce l'insieme dei termini in maniera induttiva come segue:

- (i) Ogni variabile è un termine
- (ii) Ogni costante è un termine
- (iii) Se  $f^n$  è un simbolo funzionale di arità  $n$  e  $t_1, \dots, t_n$  sono termini, allora  $f^n(t_1, \dots, t_n)$  è un termine

un termine che non contiene variabili si dice **chiuso**.

**Definizione 7.2** (Formule ben formate). Dato un linguaggio del primo ordine  $\mathcal{L}$ , si definisce l'insieme delle formule ben formate in maniera induttiva come segue:

1. Formule atomiche ben formate:
  - (i) Se  $P^n$  è un simbolo di predicato di  $\mathcal{L}$  e  $t_1, \dots, t_n$  sono termini, allora  $P^n(t_1, \dots, t_n)$  è una formula atomica
  - (ii)  $\perp$  e  $\top$  sono formule ben formate
  - (iii)  $t_1 = t_2$  è una formula atomica per ogni coppia di termini  $t_1, t_2$
2. Formule ben formate:
  - (i) Se  $A$  è una formula allora  $\neg A$  è una formula
  - (ii) Se  $A$  e  $B$  sono formule, allora  $A \wedge B$ ,  $A \vee B$ ,  $A \rightarrow B$ ,  $A \leftrightarrow B$  sono formule
  - (iii) Se  $A$  è una formula e  $x$  è una variabile, allora  $\forall x A$  e  $\exists x A$  sono formule
  - (iv) Niente altro è una formula

Possiamo considerare un linguaggio del primo ordine  $\mathcal{L}$  in cui è definito il predicato binario  $A(x, y) = \text{"x è amico di y"}$ . Inoltre compariamo un riferimento a un individuo specifico "io", che formalizzo con la costante individuale  $e$ .  
La forma logica dell'enunciato diventa:

$$\forall x \forall y (A(x, y) \wedge A(y, e)) \rightarrow A(x, e)$$

x è amico di y

y è amico mio

x è amico mio

**Esempio 7.1.** Notiamo che:

- $x, y$  sono variabili
- $e$  è una costante individuale
- $A$  predicato

I termini sono:  $x, y, e$

**Definizione 7.3** (Variabili che occorrono in un termine). L'insieme  $vars(t)$  delle variabili che occorrono in un termine  $t$  è definito induttivamente come segue:

- Se  $t$  è una variabile  $x$ , allora  $vars(t) = \{x\}$
- Se  $t$  è una costante  $c$ , allora  $vars(t) = \emptyset$
- Se  $t$  è della forma  $f^n(t_1, \dots, t_n)$ , allora  $vars(t) = vars(t_1) \cup \dots \cup vars(t_n)$

Una **sottoformula** di una formula  $A$  è una parte della formula  $A$  che è essa stessa una formula. Una **occorrenza** di una variabile  $x$  in una formula  $A$  si dice **vincolata** se occorre in una sottoformula di  $A$  della forma  $\forall xA$  o  $\exists xA$ . Altrimenti si dice **libera** (se e solo se ha almeno un'occorrenza libera).

**Osservazione 7.1.** Una occorrenza di una variabile può essere sia libera che vincolata in una stessa formula.

$$A := Q(x) \rightarrow \exists xR(x, y)$$

Una formula  $A$  si dice **formula chiusa** se non ha occorrenze libere di variabili.

**Definizione 7.4** (Variabili libere e vincolate). L'insieme  $free_vars(A)$  delle variabili libere di una formula  $A$  è definito induttivamente come segue:

- Se  $A := P(t_1, \dots, t_n)$  allora  $free_vars(A) = vars(t_1) \cup \dots \cup vars(t_n)$
- Se  $A := \top$  o  $A := \perp$  allora  $free_vars(A) = \emptyset$
- Se  $A := \neg B$  allora  $free_vars(A) = free_vars(B)$
- Se  $A = A_1 \star A_2$  con  $\star$  connettivo binario, allora  $free_vars(A) = free_vars(A_1) \cup free_vars(A_2)$
- Se  $A := \forall xB$  o  $A := \exists xB$ , allora  $free_vars(A) = free_vars(B) \setminus \{x\}$

**Definizione 7.5** (Sostituzione di variabili con termini). La sostituzione di variabili con termini è un'operazione **morfologica** (cambia la forma logica di una formula del primo ordine). Sia  $s$  un termine nella logica del primo ordine. Si definisce la **sostituzione**  $s[t/x]$  della variabile  $x$  con il termine  $t$  il risultato della sostituzione di ogni occorrenza di  $x$  con  $t$ . Tale sostituzione è definita in maniera ricorsiva come segue:

1.  $s \equiv c$  con  $c$  costante, allora  $s[t/x]$  è  $s$ .
2.  $s \equiv y$  con  $y$  variabile tale che  $y \neq x$ , allora  $s[t/x]$  è di nuovo  $s$ .
3.  $s \equiv x$  :  $s[t/x]$  diventa  $t$ .
4.  $s \equiv f^n(t_1, \dots, t_n)$  :  $s[t/x]$  diventa  $f^n(t_1[t/x], \dots, t_n[t/x])$

**Definizione 7.6** (Sostituzione di variabili con termini in una formula). Sia  $E$  una formula nella logica del primo ordine. Si definisce la sostituzione  $E[t/x]$  della variabile  $x$  con il termine  $t$  il risultato della sostituzione di ogni **occorrenza libera** di  $x$  (non vincolata) con  $t$ . Tale sostituzione è definita in maniera ricorsiva come segue:

1.  $E := P^n(t_1, \dots, t_n)$  allora  $E[t/x]$  è  $P^n(t_1[t/x], \dots, t_n[t/x])$
2.  $E := t_1 = t_2$  allora  $E[t/x]$  è  $t_1[t/x] = t_2[t/x]$
3.  $E := \neg F$  allora  $E[t/x]$  è  $\neg F[t/x]$
4.  $E := F \star G$  con  $\star$  connettivo binario, allora  $E[t/x]$  è  $F[t/x] \star G[t/x]$
5.  $E := \forall yF$  allora  $E[t/x]$  è  $\forall yF[t/x]$  se  $y \neq x$  altrimenti è  $E$

Possiamo anche effettuare delle **sostituzioni simultanee** in cui sostituiamo più termini a più variabili nello stesso passaggio:

$$\theta = [t_1/x_1, \dots, t_n/x_n]$$

Ad esempio:

$$\begin{aligned}\theta &= [f(y)/x, f(x)/y] \\ p(x, y)\theta &= p(f(y), f(x))\end{aligned}$$

La sostituzione simultanea **non** coincide con l'effettuare prima una sostituzione e poi un'altra. Ad esempio:

$$\begin{aligned}\theta_1 &= [f(y)/x] & \theta_2 &= [f(x)/y] \\ p(x, y)\theta_1\theta_2 &= p(f(y), y)\theta_2 = p(f(f(x)), f(x))\end{aligned}$$

## Composizione di sostituzioni

La composizione di sostituzioni si ottiene come segue. Date le sostituzioni  $\theta = \{t_1/x_1, \dots, t_n/x_n\}$  e  $\sigma = \{u_1/y_1, \dots, u_m/y_m\}$ , la sostituzione composta  $\theta \circ \sigma$  è definita come:

$$\theta \circ \sigma = \{t_1\sigma/x_1, \dots, t_n\sigma/x_n, u_1/y_1, \dots, u_m/y_m\}$$

eliminando tutti gli elementi:

$$t_j\sigma \text{ t.c. } t_j\sigma = x_j \text{ e } u_i/y_i \text{ t.c. } y_i \in \{x_1, \dots, x_n\}$$

Ad esempio:

$$\theta = \{f(y)/x, z/y\} \quad \sigma = \{a/x, b/y, y/z\}$$

allora  $\theta \circ \sigma$  si ottiene da  $\{f(b)/x, y/y, a/x, b/y, y/z\}$  eliminando gli elementi  $a/x, y/y, b/y$ :

$$\theta \circ \sigma = \{f(b)/x, y/z\}$$

**Proprietà della composizione di sostituzioni:** per ogni espressione  $E$ :

$$E(\theta \circ \sigma) = (E\theta)\sigma$$

Nella logica proposizionale abbiamo definito il concetto di **modello** come un assegnamento di valori di verità alle variabili proposizionali.

**Definizione 7.7** (Realizzazione o struttura di un linguaggio del primo ordine). Una struttura  $\mathcal{M}$  per un linguaggio del primo ordine  $\mathcal{L}$  è costituita dagli elementi seguenti:

1. **Dominio:** un insieme  $M$  non vuoto (universo del discorso)
2. **Interpretazioni delle costanti individuali:** per ogni simbolo costante  $c$ , è un elemento  $c^{\mathcal{M}} \in M$
3. **Interpretazioni dei simboli di predicato:** per ogni simbolo predicativo  $P^n$  di arietà  $n$ ,  $n > 0$ , è una relazione  $n$ -aria definita su  $M$  ( $R^{\mathcal{M}} \subseteq M^n$ )
4. **Interpretazione di funtori:** per ogni simbolo di funzione  $f^n$ , è una funzione  $\mathcal{M}(f) : M^n \rightarrow M$  che associa a ogni  $n$ -upla di elementi di  $M$  un elemento di  $M$
5. **Interpretazione per l'uguaglianza** (vale solo per linguaggi con l'uguaglianza)

È necessario definire una corrispondenza tra ogni variabile di una formula e un elemento del dominio. Si introduce così il concetto di **assegnazione**.

**Definizione 7.8** (Assegnazione). Una assegnazione di variabili  $s$  per una struttura  $\mathcal{M}$  è una funzione che associa ad ogni simbolo di variabile un elemento del dominio  $M$  di  $\mathcal{M}$ .

Le variabili permettono di interpretare ogni **termine**

**Definizione 7.9.** Se  $t$  è un termine del linguaggio del primo ordine  $\mathcal{L}$ ,  $\mathcal{M}$  una struttura per  $\mathcal{L}$ , e  $s$  un'assegnazione di variabili per  $\mathcal{M}$ , si definisce il valore dell'assegnazione del termine  $t$   $\bar{s}(t)$  in maniera induttiva come segue:

1. Se  $t := c$  con  $c$  simbolo costante, allora  $\bar{s}(t) = c^{\mathcal{M}}$  (costanti coincidono con la loro interpretazione nella struttura)
2. Se  $t := x$  con  $x$  variabile, allora  $\bar{s}(t) = s(x)$  (interpretazione di una variabile  $x$  è il valore associato a  $x$  nell'assegnazione  $s$ )
3. Se  $t := f^n(t_1, \dots, t_n)$  con  $f^n$  funtore  $n$ -ario, allora  $\bar{s}(t) = f^{\mathcal{M}}(\bar{s}(t_1), \dots, \bar{s}(t_n))$  (interpretazione di un funtore di termini è l'interpretazione del funtore, valutata sui termini interpretati)

**Definizione 7.10. Soddisfacibilità** di una formula  $A$  in una struttura  $\mathcal{M}$  data un'assegnazione  $s$ , induttivamente nel seguente modo:

1.  $(\mathcal{M}, s) \models P(t_1, \dots, t_n)$  se e solo se  $s(t_1), \dots, s(t_n) \in P^{\mathcal{M}}$
2.  $(\mathcal{M}, s) \models \top$
3.  $(\mathcal{M}, s) \models \neg A$  se e solo se  $(\mathcal{M}, s) \not\models A$
4.  $(\mathcal{M}, s) \models A \wedge B$  se e solo se  $(\mathcal{M}, s) \models A$  e  $(\mathcal{M}, s) \models B$
5.  $(\mathcal{M}, s) \models A \vee B$  se e solo se  $(\mathcal{M}, s) \models A$  o  $(\mathcal{M}, s) \models B$
6.  $(\mathcal{M}, s) \models A \rightarrow B$  se e solo se  $(\mathcal{M}, s) \not\models A$  o  $(\mathcal{M}, s) \models B$
7.  $(\mathcal{M}, s) \models A \leftrightarrow B$  se e solo se  $(\mathcal{M}, s) \models A$  e  $(\mathcal{M}, s) \models B$  oppure  $(\mathcal{M}, s) \not\models A$  e  $(\mathcal{M}, s) \not\models B$
8.  $(\mathcal{M}, s) \models \forall x A$  se e solo se **per ogni** elemento  $d \in M$ ,  $(\mathcal{M}, s[d/x]) \models A$
9.  $(\mathcal{M}, s) \models \exists x A$  se e solo se **esiste un** elemento  $d \in M$  tale che  $(\mathcal{M}, s[d/x]) \models A$

Queste ultime due proprietà sono effettivamente aggiunte rispetto alla logica proposizionale. Si chiama semantica dei **quantificatori**.

**Definizione 7.11.** Definizione di  $(\mathcal{M}, s) \not\models A$ : si dice che una struttura  $\mathcal{M}$  non soddisfa la formula  $A$  secondo l'assegnazione  $s$  e si scrive  $(\mathcal{M}, s) \not\models A$  nel modo seguente:

1.  $(\mathcal{M}, s) \not\models P(t_1, \dots, t_n)$  se e solo se  $s(t_1), \dots, s(t_n) \notin P^{\mathcal{M}}$
2.  $(\mathcal{M}, s) \not\models \perp$  è sempre vero,  $(\mathcal{M}, s) \not\models \top$  è sempre falso
3.  $(\mathcal{M}, s) \not\models \neg A$  se e solo se  $(\mathcal{M}, s) \models A$
4.  $(\mathcal{M}, s) \not\models A \wedge B$  se e solo se  $(\mathcal{M}, s) \not\models A$  o  $(\mathcal{M}, s) \not\models B$
5.  $(\mathcal{M}, s) \not\models A \vee B$  se e solo se  $(\mathcal{M}, s) \not\models A$  e  $(\mathcal{M}, s) \not\models B$
6.  $(\mathcal{M}, s) \not\models A \rightarrow B$  se e solo se  $(\mathcal{M}, s) \models A$  e  $(\mathcal{M}, s) \not\models B$
7.  $(\mathcal{M}, s) \not\models A \leftrightarrow B$  se e solo se  $(\mathcal{M}, s) \not\models A$  e  $(\mathcal{M}, s) \models B$  oppure  $(\mathcal{M}, s) \models A$  e  $(\mathcal{M}, s) \not\models B$
8.  $(\mathcal{M}, s) \not\models \forall x A$  se e solo se esiste un elemento  $d \in M$  tale che  $(\mathcal{M}, s[d/x]) \not\models A$
9.  $(\mathcal{M}, s) \not\models \exists x A$  se e solo se per ogni elemento  $d \in M$ ,  $(\mathcal{M}, s[d/x]) \not\models A$

La scrittura  $(\mathcal{M}, s) \models A$  si legge "la formula  $A$  è vera in  $\mathcal{M}$  secondo l'assegnazione  $s$ ".

**Soddisfacibilità, verità, falsità di una formula** Una formula del primo ordine è **soddisfacibile** se esistono almeno una struttura  $\mathcal{M}$  e un'assegnazione  $s$  su  $\mathcal{M}$  tali che  $(\mathcal{M}, s) \models A$ .

Diciamo che una formula del primo ordine  $A$  è **vera** in una struttura  $\mathcal{M}$  se **per ogni** assegnazione  $s$  su  $\mathcal{M}$ ,  $(\mathcal{M}, s) \models A$ . Si scrive anche  $\mathcal{M} \models A$ .

Una formula del primo ordine  $A$  è **falsa** in una struttura  $\mathcal{M}$  se e solo se per ogni assegnazione  $s$  su  $\mathcal{M}$ ,  $(\mathcal{M}, s) \not\models A$ . Si deduce dunque che una formula può non essere né vera né falsa in una struttura.

**Tautologia, contraddizione, soddisfacibilità** Una formula del primo ordine  $A$  si dice **valida** o **tautologica** se e solo se è vera in ogni struttura.

$A$  si dice **contraddittoria** se è falsa in ogni interpretazione.

Sia  $\Phi$  un insieme di formule del primo ordine.  $\Phi$  si dice **soddisfatto** secondo l'assegnazione  $s$  se, per ogni formula  $A \in \Phi$ ,  $(\mathcal{M}, s) \models A$ .

**Definizione 7.12** (Conseguenza logica). Sia  $\Phi$  un insieme di formule del primo ordine. Si dice che  $A$  è conseguenza logica di  $\Phi$  e si scrive  $\Phi \models A$  se vale che, per ogni struttura  $\mathcal{M}$  e assegnazione  $s$ ,  $(\mathcal{M}, s) \models C$  per ogni  $C \in \Phi$  implica  $(\mathcal{M}, s) \models A$ .

Di seguito vengono enunciate le proprietà delle formule del primo ordine.

1.  $A$  è falsa in  $\mathcal{M}$  se e solo se  $\mathcal{M} \models \neg A$ .  $\mathcal{M} \models A$  se e solo se  $\neg A$  è falsa in  $\mathcal{M}$ .
2. Nessuna formula può essere contemporaneamente vera e falsa in una interpretazione.
3. Se  $\mathcal{M} \models A$  e  $\mathcal{M} \models A \rightarrow B$ , allora  $\mathcal{M} \models B$ .
4.  $A \rightarrow B$  è falsa in  $\mathcal{M}$  se e solo se  $\mathcal{M} \models A$  e  $\mathcal{M} \not\models B$ .
5.  $\mathcal{M} \models A$  se e solo se  $\mathcal{M} \models \forall x A$ . Quindi  $\mathcal{M} \models A$  se e solo se  $\mathcal{M} \models \forall x_1 \dots \forall x_n A$ . (Chiusura universale di  $A$ )
6. Se  $s(x) = s'(x)$  per ogni  $x$  che occorre libera in  $A$ , allora  $(\mathcal{M}, s) \models A$  se e solo se  $(\mathcal{M}, s') \models A$ . (**Lemma di coincidenza**)
7. Se  $A$  è chiusa, allora per ogni  $\mathcal{M}$  o  $\mathcal{M} \models A$  o  $\mathcal{M} \models \neg A$  (o tutte le assegnazioni soddisfano  $A$  oppure nessuna la soddisfa).

**Proposizione 7.1.** Per ogni formula  $A$ ,  $A$  è valida se e solo se  $\emptyset \models A$ .

*Dimostrazione.*  $\emptyset \models A$  se e solo se, per ogni  $\mathcal{M}, s$ ,  $(\mathcal{M}, s) \models A$  se e solo se per ogni  $\mathcal{M}, s$ ,  $(\mathcal{M}, s) \models A$ .  $\square$

**Proposizione 7.2.** Per ogni formula  $A$ , e per ogni insieme di formule  $\Phi$ , valgono:

1.  $\Phi \models A \iff \Phi \cup \{\neg A\}$  è insoddisfacibile
2.  $A$  è valida  $\iff \neg A$  è insoddisfacibile

**Verifica validità di una formula**  $\forall x p(x) \rightarrow p(t)$ . Assumiamo che la formula non sia valida: esiste una sua interpretazione  $\mathcal{M}$  e un'assegnazione  $s$  su  $\mathcal{M}$  tale che  $(\mathcal{M}, s) \not\models \forall x p(x) \rightarrow p(t)$ . Ovvero valgono:

$$(a) (\mathcal{M}, s) \models \forall x p(x)$$

$$\iff \text{per ogni } d \in D, (\mathcal{M}, s[d/x]) \models p(x)$$

$$\iff \text{per ogni } d \in D, s[d/x](x) \in \mathcal{M}(p)$$

$$\iff \text{per ogni } d \in D, d \in \mathcal{M}(p)$$

$$(b) (\mathcal{M}, s) \not\models p(t) \text{ dunque se } s(t) = d^*, \text{ allora } d^* \notin \mathcal{M}(p)$$

E si raggiunge la contraddizione.

**Verifica della NON validità di formule** Un modo per verificare la non validità di una formula è quello di cercare un contromodello, ovvero una struttura in cui la formula non è vera.

Sia  $(\forall x p(x) \rightarrow \forall x q(x)) \rightarrow \forall x (p(x) \rightarrow q(x))$  (Cerco una struttura e una assegnazione in cui la premessa è vera e la conclusione è falsa).

$\forall x p(x) \rightarrow \forall x q(x)$  è sicuramente vera quando la premessa  $\forall x p(x)$  è falsa.

Definiamo un dominio  $D = \{0, 1\}$  e le interpolazioni di  $p$  e  $q$  in modo che  $\forall x p(x)$  sia falsa e  $\forall x (p(x) \rightarrow q(x))$  sia falsa.

Definiamo una struttura  $\mathcal{M}$  con dominio  $D$ . Sia  $p^{\mathcal{M}} = \{p(0)\}$  e  $q^{\mathcal{M}} = \emptyset$ .

In questa struttura,  $\forall x p(x)$  è falsa, dunque  $\forall x p(x) \rightarrow \forall x q(x)$  è vera.

Inoltre,  $\forall x (p(x) \rightarrow q(x))$  è falsa in  $\mathcal{M}$  perché se  $x = 0$  ho che  $p(0)$  è vera, ma  $q(0)$  è falsa.

Dunque abbiamo trovato un contromodello per la formula e dunque non è valida.



**Metodi per dimostrare validità/non validità/conseguenza logica** Per dimostrare che:

- La formula  $A$  è valida: per assurdo si assume che esista un contromodello  $(\mathcal{M}, s)$  di  $A$  e si giunge ad un assurdo.
- La formula  $A$  non è valida: si trova un contromodello  $(\mathcal{M}, s) \models A$ .

Per dimostrare che:

- $S \models A$ 
  - per assurdo: si assume che esistano un'interpretazione  $\mathcal{M}$  e un'assegnazione  $s$ , tali che  $(\mathcal{M}, s) \models S$  e  $(\mathcal{M}, s) \not\models A$  e si giunge ad un assurdo.
  - dimostrazione diretta: si assume che  $\mathcal{M}$  e un'assegnazione  $s$ , tali che  $(\mathcal{M}, s) \models S$  e si dimostra che  $(\mathcal{M}, s) \models A$ .
- $S \not\models A$ : si costruisce un contromodello, cioè un'interpretazione  $\mathcal{M}$  e un'assegnazione  $s$ , tali che  $(\mathcal{M}, s) \models S$  e  $(\mathcal{M}, s) \not\models A$ .

**Definizione 7.13** (Equivalenza Logica). Due formule  $A$  e  $B$  si dicono **logicamente equivalenti** se per ogni  $\mathcal{M}, s$   $(\mathcal{M}, s) \models A \iff (\mathcal{M}, s) \models B$ .

**Proposizione 7.3.** Due formule  $A$  e  $B$  sono logicamente equivalenti se e solo se  $\models A \leftrightarrow B$ .

Osserviamo che il linguaggio della logica del primo ordine può essere espresso in maniera equivalente utilizzando uno solo dei quantificatori, infatti ognuno dei due quantificatori può essere espresso utilizzando la **negazione** e l'**altro quantificatore**.

**Osservazione 7.2.** Un linguaggio del primo ordine può essere formulato utilizzando solamente i connettivi  $\rightarrow, \neg$  e il quantificatore  $\forall$ . (Analogo al caso della logica proposizionale, con l'aggiunta del quantificatore).

## 7.2 Sistema di inferenza per la logica del primo ordine

Nel contesto della logica del primo ordine le definizioni **derivazione** e **teorema** della logica proposizionale possono essere estese. Ricordiamo che un sistema di inferenza è costituito da un insieme di assiomi e un insieme di regole di inferenza.

## 7.3 Sistema Hilbertiano per la logica del primo ordine

Sia dato un linguaggio con i connettivi  $\rightarrow, \neg, \forall$ . Si considerano i seguenti assiomi:

1.  $A \rightarrow (B \rightarrow A)$
2.  $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$
3.  $(\neg B \rightarrow \neg A) \rightarrow (((\neg B \rightarrow A) \rightarrow B))$
4.  $\forall x A \rightarrow A[x/t]$  se  $t$  è sostituibile a  $x$  in  $A$
5.  $\forall x(A \rightarrow B) \rightarrow ((\forall x A) \rightarrow (\forall x B))$  se  $A$  non ha occorrenze libere di  $x$

e le regole di inferenza:

1. Modus Ponens:  $\frac{A, A \rightarrow B}{B}$
2. Generalizzazione:  $\frac{A}{\forall x A}$

**Teorema 7.1** (Teorema di deduzione). Siano  $A, B$  formule del primo ordine e  $\Phi$  un insieme di formule.  $\Phi \vdash_{HAL} B$  se e solo se  $\Phi \vdash_{HAL} A \rightarrow B$ .

Il sistema Hilbertiano è **corretto** (ogni formula derivabile è conseguenza logica della KB) e **completo** (ogni conseguenza logica della KB è derivabile dal sistema) nella logica del primo ordine.

## 7.4 Teorie del primo ordine

Si parla di **teoria del primo ordine** in riferimento a una struttura composta da **linguaggio del primo ordine** e **sistema di inferenza** (assiomi logici, regole di inferenza, assiomi propri - specifici per il dominio di interesse).

### Teorie del primo ordine con uguaglianza

Le teorie del primo ordine con uguaglianza sono caratterizzate dalla presenza del simbolo  $=$  tra i simboli di predicato.  $= \in \mathcal{P}$ . In una generica **teoria dell'uguaglianza** si considerano i seguenti assiomi propri:

1.  $\forall x(x = x)$  (riflessività)
2.  $\forall x\forall y(x = y \rightarrow (A \rightarrow \hat{A}))$  dove  $\hat{A}$  è ottenuta sostituendo alcune occorrenze libere di  $x$  con  $y$

**Osservazione 7.3** (Consistenza). *Diamo una definizione di per i piani di lettura **sintattico** e **semantico**:*

- **Piano semantico:** insieme di enunciati  $\Phi$  si dice **consistente** se esiste almeno una struttura in cui tutti gli enunciati di  $\Phi$  sono veri.
- **Piano sintattico:** insieme di enunciati  $\Phi$  si dice **consistente** se da questo non sono derivabili contraddizioni.

**Definizione 7.14** (Teoria del primo ordine). Una teoria del primo ordine è una struttura formata da un linguaggio del primo ordine e un insieme di assiomi logici e regole di inferenza.

**Gli alberi di Beth** sono un metodo di dimostrazione automatica che vale anche per la logica dei predicati, ma esistono delle limitazioni. Le regole per la costruzione degli alberi di formule riferite ai connettivi proposizionali non cambiano, ma vanno estese per gestire i quantificatori. Queste ultime si raggruppano nelle seguenti forme logiche:

- $\forall xC$  (enunciato universale vero)
- $\neg\forall xC$  (enunciato universale falso)
- $\exists xC$  (enunciato esistenziale vero)
- $\neg\exists xC$  (enunciato esistenziale falso)

| Tabella 5.1: Regole per gli alberi di Beth / quantificatori |                   |                                             |                   |                   |  |
|-------------------------------------------------------------|-------------------|---------------------------------------------|-------------------|-------------------|--|
| ESEMPLIFICAZIONE                                            |                   |                                             | ESPOSIZIONE       |                   |  |
| universale:                                                 | [ $\forall$ ]     | $\forall z D$                               | [ $\neg\forall$ ] | $\neg\forall z D$ |  |
|                                                             |                   | $\vdots$                                    |                   | $\vdots$          |  |
|                                                             |                   | $D[z := e_1], \dots, D[z := e_n]$           |                   | $\neg D[x := e]$  |  |
|                                                             |                   | $e_1, \dots, e_n$                           |                   | $e$               |  |
| costanti disponibili                                        |                   |                                             | costante nuova    |                   |  |
| esistenziale:                                               | [ $\neg\exists$ ] | $\neg\exists z D$                           | [ $\exists$ ]     | $\exists z D$     |  |
|                                                             |                   | $\vdots$                                    |                   | $\vdots$          |  |
|                                                             |                   | $\neg D[z := e_1], \dots, \neg D[z := e_n]$ |                   | $D[z := e]$       |  |
|                                                             |                   | $e_1, \dots, e_n$                           |                   | $e$               |  |
| costanti disponibili                                        |                   |                                             | costante nuova    |                   |  |

Precisiamo cosa significa dire che un albero è **chiuso**. Nella logica dei predicati, un albero è chiuso quando:

- Tutte le formule del tipo  $[\rightarrow]$ ,  $[\neg \rightarrow]$ ,  $[\wedge]$ ,  $[\neg \wedge]$ ,  $[\vee]$ ,  $[\neg \vee]$ ,  $[\exists]$ ,  $[\neg \exists]$ , che sono su un ramo aperto sono state analizzate con la regola corrispondente
- Tutte le formule del tipo  $[\forall]$ ,  $[\neg \exists]$  che sono su un ramo aperto dell'albero sono state esemplificate ed eventualmente riesemplificate con tutte le costanti presenti nell'albero

**Osservazione 7.4.** *Con gli alberi di Beth, partendo dalla negazione di una formula di interesse, costituiscono una procedura algoritmica per verificare se una formula è tautologica o meno.*

*Nel caso in cui l'albero sia **aperto**, ossia nel caso esista un **controesempio**, questo controesempio può essere estratto.*

Per estrarre un controesempio da un albero di Beth del primo ordine si procede nel seguente modo:

1. Selezionare un ramo aperto
2. Definire un **dominio** con **tutte e sole le costanti individuali** che compaiono lungo il ramo ( $D = \{a_1, \dots, a_n\}$ )
3. Significato di un predicato unario  $P(\star) : P^M = \{P(a_i) \text{ t.c. } P(a_i) \text{ compare nel ramo e } \neg P(a_i) \text{ non compare nel ramo}\}$
4. Significato di un predicato  $P^{(n)M} = \{P(c_{i1}, \dots, c_{in}) \text{ t.c. } P(c_{i1}, \dots, c_{in}) \text{ compare nel ramo e } \neg P(c_{i1}, \dots, c_{in}) \text{ non compare nel ramo}\}$

## Decidibilità della logica dei predicati

Una logica è **decidibile** se il problema di determinare se  $S \models A$  è risolvibile in tempo finito. Ovvero termina con output **TRUE** se  $S \models A$  e **FALSE** altrimenti.

- La **logica proposizionale** è **decidibile**
- La **logica del primo ordine** è **indecidibile** (ci si riconduce al problema della fermata)

Consideriamo un sistema assiomatico **corretto** e **completo** per la logica dei predicati in cui la proprietà di essere un assioma e la relazione di derivabilità siano decidibili. Se il linguaggio è numerabile, allora il sistema di inferenza è **semidecidibile**.

- Se  $S \models A$ , allora il processo prima o poi termina, poiché si troverà una derivazione
- Se  $S \not\models A$ , allora il processo potrebbe non terminare

**Osservazione 7.5.** *Distinguiamo tra **completezza** e **decidibilità**.*

- **Completezza:** *un sistema è completo se ogni formula valida ammette una dimostrazione*
- **Decidibilità:** *riguarda lo stabilire se una formula **arbitraria** sia valida e dimostrabile*

**Osservazione 7.6.** *Esistono casi di linguaggi del primo ordine che sono **decidibili**: calcolo monadico, formule puramente universali, formule puramente esistenziali.*

## Unificazione

L'operazione di unificazione permette di implementare e applicare algoritmi di deduzione automatica. Un **unificatore** per un insieme di espressioni  $\{E_1, \dots, E_k\}$  è una sostituzione  $\theta$  che rende uguali le formule:

$$E_1\theta = E_2\theta = \dots = E_k\theta$$

Se esiste un unificatore per l'insieme  $\{E_1, \dots, E_k\}$ , allora l'insieme si dice **unificabile**.

Un unificatore  $\theta$  è un **unificatore più generale (mgu)** se per ogni altro unificatore  $\sigma$  di  $\{E_1, \dots, E_k\}$  esiste una sostituzione  $\lambda$  tale che  $\theta = \sigma \circ \lambda$ .

Un mgu è dunque un unificatore più conveniente.

**Algorithm 2** Algoritmo di unificazione di Robinson**Require:** Termini o predicati  $A, B$ **Ensure:** Sostituzione unificante  $\sigma$ , oppure **fail**1:  $A_0 \leftarrow A, \quad B_0 \leftarrow B$ 2:  $\sigma_0 \leftarrow \emptyset$  {Sostituzione iniziale vuota}3:  $i \leftarrow 0$ **LOOP** {Ciclo principale finché si ottiene una soluzione o un fallimento}4: **if**  $A_i = B_i$  **then**5:     **return**  $\sigma_i$  {Espressioni identiche: ho finito}6: **end if**7: Seleziona il *disagreement set*  $\{e, e'\}$  tra  $A_i$  e  $B_i$ 8: **if** esiste  $x_i \in \{e, e'\}$  tale che  $x_i$  è variabile e l'altro termine  $e_i$  non contiene  $x_i$  **then**

9:     Definisci la nuova sostituzione parziale:

$$\sigma_{i+1} \leftarrow \sigma_i \circ \{e_i/x_i\}$$

10:   Applica la sostituzione ad  $A_i$  e  $B_i$ :

$$A_{i+1} \leftarrow A_i\{e_i/x_i\}, \quad B_{i+1} \leftarrow B_i\{e_i/x_i\}$$

11:    $i \leftarrow i + 1$ 12: **else**13:     **return fail**  $\{\{e, e'\}$  non è unificabile (occorrenza della variabile o clash di simboli) $\}$ 14: **end if****END LOOP****Algoritmo di unificazione di Robinson per mgu**

**Definizione 7.15** (Disagreement set). Siano  $A, B$  due termini e sia  $k$  la posizione più a sinistra in cui le due sequenze di simboli differiscono. L'insieme  $\{e_1, e_2\}$  delle due espressioni che iniziano alla posizione  $k$  in  $A$  e  $B$  si dice **disagreement set**.

**Esempio 7.2.** Il disagreement set di  $A = p(x, f(c), x)$  e  $B = p(x, g(y, a), z)$  è dato da  $\{f(c), g(y, a)\}$ .

**Osservazione 7.7.** Nel caso peggiore l'algoritmo di Robinson può avere complessità esponenziale, ma nella pratica esistono risultati che mostrano che l'algoritmo di Robinson è implementabile in tempo polinomiale.

La risoluzione proposizionale si applica a clausole: vediamo ora i passi per ottenere una forma a clausole in logica dei predicati.

**Definizione 7.16** (Forma normale prenessa). Una formula del primo ordine è in **forma normale prenessa** se è della forma

$$Q_1x_1 \dots Q_nx_n(A)$$

dove  $Q_i$  sono quantificatori e in  $A$  non compaiono quantificatori.

**Definizione 7.17** (Forma normale di Skolem). Una formula del primo ordine è in **forma normale di Skolem** se è della forma

$$\forall x_1 \dots \forall x_n A$$

dove  $A$  è una formula senza quantificatori.

Si dimostra che ogni formula del primo ordine è logicamente equivalente a una formula in forma normale prenessa. Inoltre la trasformazione in forma di Skolem (**skolemizzazione**) si può effettuare nel seguente modo:

1. Trasformazione in forma prenessa

## 2. Eliminazione dei quantificatori esistenziali

$$\forall x_1 \dots \forall x_n \exists y A \Rightarrow \forall x_1 \dots \forall x_n A[f(x_1, \dots, x_n)/y]$$

con  $f$  che è un **nuovo simbolo** detto **funzione di Skolem**. Ovvero sostituisco le occorrenze di  $y$  con un termine che dipende dalle variabili quantificate universalmente che precedono l'esistenziale.

$$\begin{array}{ccc} \exists x \forall y \forall z \exists u \forall w \exists v p(x, y, z, u, w, v) & \xrightarrow{\quad} & \forall y \forall z \exists u \forall w \exists v p(c, y, z, u, w, v) \\ \text{(forma prenessa)} & & \\ & & \forall y \forall z \forall w \exists v p(c, y, z, f(y, z), w, v) \\ & & \downarrow \text{L'esistenziale su } v \text{ dipende da } y, z, w \\ & & \forall y \forall z \forall w p(c, y, z, f(y, z), w, g(y, z, w)) \end{array}$$

Si sono introdotti i simboli nuovi  $c, f, g$  seguendo i ragionamenti sul fatto che il simbolo quantificato dall'esistenziale dipende dai quantificatori che lo precedono.

È possibile, data una formula del primo ordine  $A$ , trasformarla in una **forma a clausole** con il seguente procedimento:

1. Trasformare  $A$  in forma normale di Skolem
2. Eliminazione dei quantificatori universali
3. Trasformazione della matrice (formula priva di quantificatori) in **CNF**
4. Trasformazione in un insieme di clausole

Osserviamo che:

1. Se  $pre(A)$  è una formula in forma prenessa di  $A$ , allora  $A \equiv pre(A)$  (formula è logicamente equivalente alla sua forma prenessa)
2. Se  $CNF(A)$  è una forma normale congiuntiva di  $A$ , allora  $\forall x_1 \dots \forall x_n A \equiv \forall x_1 \dots \forall x_n CNF(A)$
3.  $A$  sta per  $\forall x_1 \dots \forall x_n A$  (eliminazione dei quantificatori universali)
4.  $\forall x_1 \dots \forall x_n (C_1 \wedge \dots \wedge C_n) \equiv \forall x_1 \dots \forall x_n C_1 \wedge \dots \wedge \forall x_1 \dots \forall x_n C_n$

**Osservazione 7.8.** Una formula **non** è logicamente equivalente alla sua forma di Skolem.

**Osservazione 7.9.** La skolemizzazione **preserva la soddisfacibilità**. Dunque se una formula è soddisfacibile, allora lo è anche la sua forma di Skolem, mentre se la formula di Skolem non è soddisfacibile, allora non lo è neanche la formula originale.

**Teorema 7.2.**  $A$  è insoddisfacibile se e solo se la sua forma a clausole è insoddisfacibile. Se  $\Phi$  è un insieme di formule e  $\Phi'$  è ottenuto trasformando ogni formula di  $\Phi$  in forma a clausole, allora  $\Phi$  è insoddisfacibile se e solo se  $\Phi'$  è insoddisfacibile.

Ha dunque senso introdurre un sistema di inferenza per clausole da utilizzare come metodo di refutazione.

**Definizione 7.18** (Risoluzione come metodo di refutazione (primo ordine)). La risoluzione controlla l'insoddisfacibilità di un insieme di clausole. La **regola di risoluzione** per clausole è:

$$\frac{C_1 \cup \{P\}; C_2 \cup \{\neg Q\}}{C_1 \theta \cup C_2 \theta}$$

dove  $\theta$  è un **mgu** per  $P$  e  $Q$ .

**Fattorizzazione nel primo ordine**

Nella logica proposizionale due letterali sono ridotti a uno tramite fattorizzazione se questi sono **identici**, mentre nel primo ordine due letterali sono ridotti a uno tramite fattorizzazione se questi sono **unificabili**. Nella formulazione della risoluzione le clausole derivabili hanno due fattori: se si aggiunge la possibilità di fattorizzare si include la derivazione di clausole con un solo fattore, e, in particolare, della clausola vuota.